

班 级 2203054w

学 号 22009201185

西安电子科技大学

本科毕业设计论文



题 目 智能客服机器人的设计与交互优化

学 院 计算机科学与技术学院

专 业 计算机科学与技术

学生姓名 王奇

导师姓名 魏昱恒、Harris H M XIE

毕业设计（论文）诚信声明书

本人声明：本人所提交的毕业论文《智能客服机器人的设计与交互优化》是本人在指导教师指导下独立研究、写作的成果，论文中所引用他人的无论以何种方式发布的文字、研究成果，均在论文中加以说明；有关教师、同学和其他人员对本文的写作、修订提出过并为我在论文中加以采纳的意见、建议，均已在我的致谢辞中加以说明并深致谢意。

本论文和资料若有不实之处，本人承担一切相关责任。

论文作者：_____（签字） 时间： 年 月 日

指导教师已阅：_____（签字） 时间： 年 月 日

摘要

个人银行业务迁移到网页端和移动端之后，余额查看、交易流水筛选、收支统计、常见问题咨询等操作仍然依赖多个页面和表单条件；静态 FAQ 也难以处理与账户、流水相关的实时数据。与此同时，余额、交易明细和转账属于敏感业务事实，若由大语言模型直接生成金额或触发资金操作，容易出现事实失真和误执行风险。围绕这些问题，本文设计并实现了面向个人银行业务的智能客服机器人原型 BankCopilot。系统前端采用 Vue 3、Element Plus 和 ECharts，后端采用 Spring Boot、MyBatis 和 MySQL，并接入通义千问模型完成自然语言意图解析与分析性回复组织。本文将系统职责划分为“大模型负责理解与表达，后端确定性服务负责事实与执行控制”：模型把用户输入转换为余额查询、流水查询、账单统计、FAQ 咨询或转账等结构化意图，账户信息、流水明细、统计结果和转账执行则全部由后端服务和数据库给出。系统最终实现了文本输入、语音转文字、AI 查余额、AI 查流水、AI 账单统计与分析、AI 周报/月报、FAQ 知识库问答、异常情绪安抚、人工客服引导以及 AI 智能转账二次确认，并在交互上加入 SSE 流式响应、处理过程展示、统计图表、快捷操作和导出链接。测试阶段，21 项后端自动化测试全部通过；80 条意图与分流样本的通过率为 95.0%；FAQ 知识库可以对银行相关问题返回标准答案或相似推荐，并把无关问题引入未命中分支；9 名匿名测试者在本地原型和虚拟演示数据环境中产生 81 条任务聊天记录，核心任务均完成。测试结果说明，BankCopilot 在不让模型直接决定业务事实和资金变动的前提下，可以压缩高频查询路径，补充常见问题分流，并增强银行智能客服交互过程的可解释性和可控性。

关键词： 智能客服机器人 大语言模型 自然语言处理 银行业务 账单分析
交互优化 二次确认

ABSTRACT

As personal banking moves to web and mobile channels, users still need to pass through several pages and filtering forms when checking balances, browsing transaction records, reviewing income and expenditure, or asking routine service questions. A static FAQ module also cannot answer questions that depend on real account and transaction data. In addition, balances, transaction details, and fund transfers are sensitive business facts, so allowing a large language model to directly produce these facts or trigger operations may introduce hallucinated results and mistaken execution. This thesis therefore designs and implements BankCopilot, an intelligent customer service robot prototype for personal banking. The front end is built with Vue 3, Element Plus, and ECharts, while the back end uses Spring Boot, MyBatis, and MySQL. The system also integrates the Qwen model for intent parsing and analytical response generation. The main design idea is to separate model understanding from deterministic business execution: the model converts user utterances into structured intents such as balance inquiry, transaction inquiry, bill statistics, FAQ consultation, or transfer, while account data, transaction records, statistical results, and transfer execution are handled by back-end services and the database. BankCopilot supports text input, speech-to-text input, balance inquiry, transaction inquiry, bill statistics and analysis, weekly or monthly briefs, FAQ knowledge-base answering, emotional support, human-service guidance, and two-step confirmation for intelligent transfer. Its interaction design includes SSE streaming responses, process tracing, statistical charts, quick actions, and export links. In testing, all 21 back-end automated test cases passed; 80 intent and routing samples achieved a pass rate of 95.0%; the FAQ knowledge base returned standard answers or similar recommendations for banking-related questions and routed irrelevant questions to the no-answer branch; and nine anonymous testers produced 81 task chat records in a local prototype environment with virtual demonstration data, with all core tasks completed. These results suggest that BankCopilot can shorten high-frequency inquiry paths, supplement FAQ routing, and improve the explainability and controllability of intelligent customer service without letting the model directly determine business facts or fund movements.

Keywords: intelligent customer service robot large language model natural language processing banking service bill analysis interaction op-

timization two-step confirmation

目录

摘 要	i
ABSTRACT	iii
目录	v
第一章 绪论	1
1.1 研究背景与问题提出	1
1.2 课题目标、研究内容与创新点	3
1.3 本文组织结构	5
第二章 相关研究与行业实践分析	7
2.1 传统银行管理系统与核心银行系统	7
2.2 银行 AI 应用的行业实践	7
2.3 银行 AI 系统面临的关键问题	8
2.4 本文系统与行业系统的差异	9
2.5 课题痛点与本文优化措施	10
第三章 相关技术与理论基础	13
3.1 大语言模型与自然语言意图识别	13
3.2 自然语言处理方案选择与比较	13
3.3 Spring Boot 后端技术	15
3.4 Vue 与前端交互技术	15
3.5 数据持久化与统计分析技术	16
第四章 系统需求分析	19
4.1 系统总体需求	19
4.2 AI 核心功能需求	20
4.3 非功能性需求	20
第五章 系统总体设计	23
5.1 系统架构设计	23
5.2 AI 模块设计	24
5.3 数据库设计	25
5.4 接口与交互设计	27
第六章 AI 智能客服核心功能设计与实现	29
6.1 AI 请求处理总体流程	29

6.2 AI 语音输入功能实现	31
6.3 AI 情绪安抚与人工客服引导功能实现	31
6.4 FAQ 知识库问答功能实现	32
6.5 AI 查余额功能实现	33
6.6 AI 查流水功能实现	33
6.7 AI 账单统计与分析功能实现	35
6.8 AI 周报/月报功能实现	38
6.9 AI 智能转账功能实现	39
第七章 系统测试与交互优化分析	45
7.1 测试环境与测试方法	45
7.1.1 测试指标定义与计算方法	46
7.2 后端自动化测试与指标记录	48
7.3 意图识别准确性测试	50
7.4 功能测试	53
7.5 用户体验测试与反馈分析	57
7.6 交互优化设计分析	62
7.7 测试结论	64
第八章 结论与展望	65
8.1 主要结论	65
8.2 实践价值	66
8.3 不足与展望	66
第九章 结束语	69
致谢	71
参考文献	73
附录 A 主要接口与数据结构	75
A.1 AI 聊天请求结构	75
A.2 AI 聊天响应结构	75
A.3 统计查询结构	75
A.4 测试样本说明	76
A.5 匿名反馈测试日志明细	76

第一章 绪论

1.1 研究背景与问题提出

银行个人业务已经大量迁移到移动端和网页端，但许多高频操作仍然以菜单、表单和列表为主。用户想看余额、筛选交易流水、了解某段时间的收支，或者发起一笔转账时，往往需要先找到对应页面，再填写时间、方向、金额等条件。此类需求重复出现，完全依靠人工客服既会增加服务压力，也会延长用户等待时间。因此，本课题关注的不是泛化闲聊机器人，而是能听懂用户日常表达、再连接银行业务服务的智能客服入口。

大语言模型提升了客服系统理解口语化表达和组织回复的能力，使传统基于规则或检索的问答逐渐转向可处理多轮上下文的智能助手^[1]。不过，银行场景与普通咨询不同：账户余额、流水记录和转账结果都必须有明确的数据来源，不能由模型按照语言习惯推测。基于这一限制，本课题把大模型放在“入口”和“表达”的位置，让它解析用户意图、整理说明文字，而把查询、统计和资金变动交给后端确定性服务处理，以此兼顾交互灵活性和业务安全。

BankCopilot 以个人账户业务为对象，提供一个可在银行管理系统中使用的 AI 助手。为避免后文表述不一致，本文中的“客服机器人”“智能客服助手”和“AI 助手”均指 BankCopilot；其中“AI 助手”更多用于说明前端聊天面板及其交互入口。系统允许用户通过文本或语音转文字提出请求，可完成余额查询、流水查询、账单统计、周报/月报生成和智能转账确认等操作。与只依赖静态问答库的客服系统相比，BankCopilot 的回复会经过意图解析、业务服务调用和结果封装三个步骤，最终把账户、流水和统计服务返回的数据整理为文本、图表或报表导出链接。

在银行系统中，智能客服不能只被看作聊天窗口，它同时包含业务入口、流程编排和风险控制。以流水查询为例，传统页面通常要求用户进入交易流水页，选择收入或支出，填写起止日期，再调整分页或条数后查看结果。若换成客服交互，用户可能只输入“查最近 5 条支出记录”或“看最近 7 天支出趋势”。此时系统要做的事情，是把这类口语表达转换为后端查询条件，而不是单纯生成一段听起来合理的回答。也就是说，自然语言理解必须和业务规则约束一起工作，智能客服才可能真正成为银行服务入口。

银行业务还对准确性和安全性提出了更高要求。余额、流水和转账结果都关

系到用户资金感知，系统不能让模型自行补全或编造金额。本文采用的做法是：模型负责理解“用户想做什么”和把结果解释清楚，数据库查询、统计聚合以及资金变动仍由传统后端服务完成。这样既利用了大模型处理自然语言的优势，又让数据来源、接口调用和业务状态可以回溯。这个职责划分，也是本文系统区别于普通聊天机器人的关键所在。

据此，本文把课题问题归纳为四点。其一，传统银行页面依赖菜单和筛选表单，余额查询、流水筛选和账单统计等操作路径偏长。其二，普通 FAQ 只能回答固定问题，无法处理依赖账户、流水和统计结果的实时业务。其三，大语言模型适理解表达，但不能成为余额、流水、统计金额和转账结果的事实来源。其四，转账类操作风险较高，参数解析、后端校验和用户确认之间必须划出明确边界。因此，本文要解决的不是“做一个会聊天的机器人”，而是在银行业务原型中设计一条可解释、可控制、可验证的 AI 辅助操作链路。

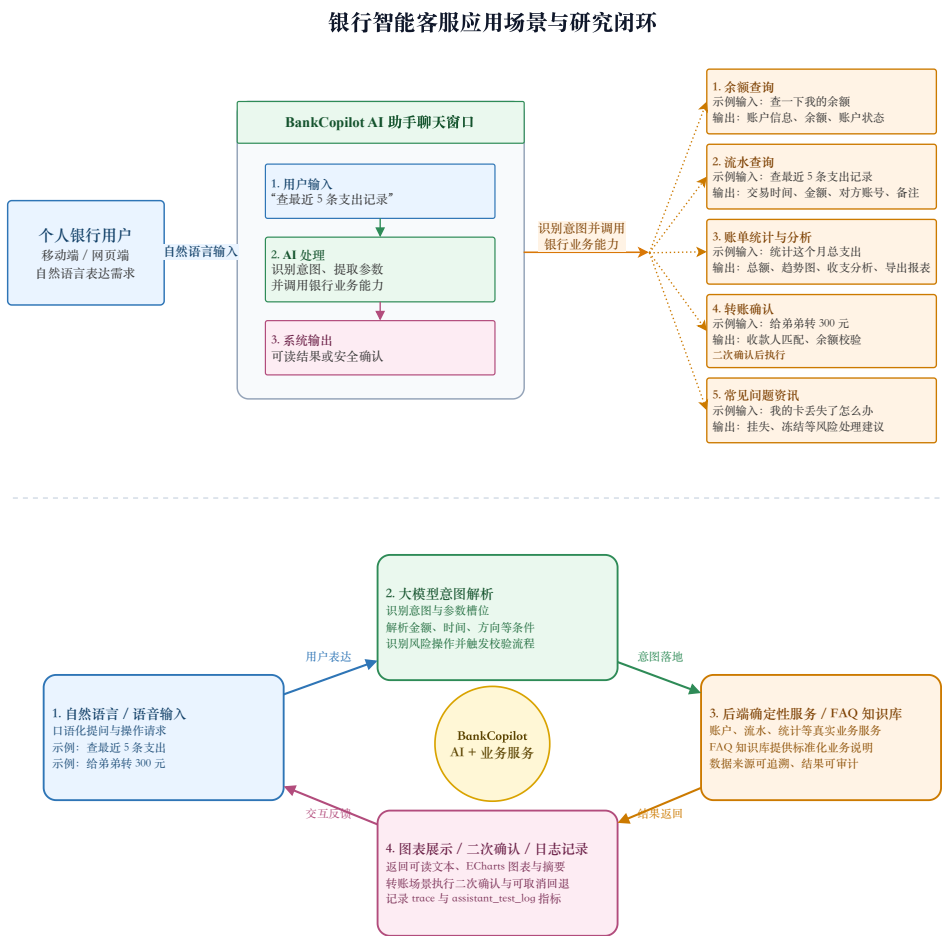


图 1.1 银行智能客服应用场景与研究闭环示意图

1.2 课题目标、研究内容与创新点

本课题的总体目标，是在已有银行管理系统的业务基础上实现一个智能客服机器人原型。研究重点放在五个方面：自然语言入口如何设计，业务意图如何解析，多轮状态如何保存，AI 助手如何调用账户、流水、统计和转账服务，以及前端如何把处理过程展示给用户。系统以文本输入为主，同时加入语音转文字入口，用于覆盖个人银行业务中的高频查询和确认场景。

结合任务书要求和项目源码，本文的研究内容按实现链路展开。

首先，根据银行个人业务场景划定 AI 助手的功能边界和交互流程；其次，比较自然语言处理方案，并使用通义千问模型把用户输入转换为结构化意图；再次，在服务层设计待确认和待补全状态，支撑转账确认、流水查询补月份等多轮交互；随后，把 AI 助手接入账户、流水、统计和转账服务，使回复由真实业务数据驱动；最后，在前端补充语音输入、SSE 流式响应、处理过程追踪、图表展示、快捷操作和报表导出等交互能力。

表 1.1 课题研究内容与系统实现对应关系

研究内容	本文实现方式
自然语言理解	通过 QwenClient 调用通义千问，将用户输入解析为 intent 字段、时间范围、金额、收款人等结构化字段。
对话管理	通过 pendingMap 和 pendingTxnQueryMap 维护待确认转账和待补全流水查询状态。
业务系统集成	调用 AccountService、TransactionService、StatisticsService 和 PayeeService 完成真实业务处理。
交互优化	前端 AssistantPanel 实现语音输入、SSE 流式响应、AI 过程展示、图表展示、快捷确认与导出功能。
安全控制	转账前校验账户、余额和收款人信息，并要求用户二次确认。

这些内容共同组成了 BankCopilot 的 AI 能力闭环：自然语言理解把原始输入转成可执行意图，对话管理处理缺失信息和风险确认，业务服务集成保证结果来自数据库，交互优化则把过程和结果以更清楚的形式呈现给用户。后续章节也将沿着这条闭环展开，尽量把 AI 功能落到接口、服务、数据表和测试记录上，而不是只停留在概念描述。

表 1.2 本文创新点与后文章节对应关系

创新点或优化点	解决的问题	具体实现位置	验证位置
自然语言业务入口与语音转文字输入	传统银行页面依赖菜单、表单和筛选条件，用户完成查询路径较长。	5.4 接口与交互设计；6.1 AI 请求处理总体流程；6.2 AI 语音输入功能实现。	7.4 功能测试 T1、T2、T11；7.5 用户体验测试 U1、U2、U5。
大模型与后端确定性服务分工	大模型可能编造余额、流水、统计金额或业务结论。	3.1 大语言模型与自然语言意图识别；5.1 系统架构设计；5.2 AI 模块设计；6.5–6.8 查询与统计功能实现。	7.3 意图识别准确性测试；7.4 功能测试；7.7 测试结论。
FAQ 知识库与查询日志	静态客服或纯大模型回答难以保证常见银行问题答案稳定、可维护。	5.3 数据库设计中的 <code>faq_knowledge</code> 、 <code>faq_query_log</code> ；6.4 FAQ 知识库问答功能实现。	7.4 FAQ 检索统计；附录 A.4、A.5、A.6、A.7。
多轮对话状态管理	用户输入缺少月份、确认、取消等上下文时，单轮问答无法正确处理。	5.2 AI 模块设计；6.1 AI 请求处理总体流程；6.6 AI 查流水多轮补全；6.9 AI 智能转账。	7.4 功能测试 T3、T7、T8、T9。
统计解释、图表展示与简报	传统流水表格需要用户自行理解和计算，统计结果不直观。	5.4 接口与交互设计；6.7 AI 账单统计与分析；6.8 AI 周报/月报。	7.4 功能测试 T4、T5、T6；7.5 用户体验测试 U6；7.6 交互优化设计分析。
转账二次确认、账号脱敏和取消回退	AI 直接执行资金操作存在误操作风险。	4.3 非功能性需求；6.9 AI 智能转账功能实现；表 6.5 安全控制设计。	7.4 功能测试 T7、T8、T9；7.5 用户体验测试 U7。
过程展示、统计口径说明和能力边界控制	用户难以判断 AI 是否真正调用业务服务，也难以识别模型是否超出能力范围。	5.2 AI 模块设计；6.1 响应结构 <code>trace/explain</code> ；6.7 统计 <code>explain/exportUrl</code> ；7.6 交互优化设计分析。	7.4 功能测试 T10；7.6 交互优化设计分析；7.7 测试结论。
匿名测试日志与反馈优化闭环	无实际用户反馈缺乏对改进方向	5.3 <code>assistant_test_log</code> 数据表；7.1–7.5 测试环境、日志说明和用户体验测试；附录 A.7。	7.5 用户体验测试与反馈分析；附录 A.8–A.10。

表 1.2 把本文的创新点和后续章节对应起来。可以看到，相关工作不是单独写成口号，而是分别落在架构设计、AI 模块、数据表、功能代码和测试验证中。第五章回答系统如何设计，第六章说明功能如何落地，第七章再通过意图识别、功能测试、FAQ 检索统计和匿名用户体验测试检查这些设计是否有效。

本文的创新主要落在工程集成和交互设计上。第一，BankCopilot 不再把客服入口限制为固定问答，而是让用户通过文本或语音转文字发起余额查询、流水筛选、账单统计、FAQ 咨询和转账确认。第二，系统采用“大模型理解与表达、后端确定性服务提供业务事实”的分工，Qwen 模型主要做意图解析和分析性语言生成，账户、流水、统计和转账结果由 Spring Boot 后端和 MySQL 数据库提供。第三，转账流程被拆分为参数解析、后端校验、待确认、确认执行或取消回退等阶段，使高风险操作具备可确认、可取消、可追踪的链路。第四，系统建立“功能测试、日志记录、用户反馈、交互优化”的验证闭环，通过 `assistant_test_log`、自动化测试、意图测试、FAQ 检索统计和匿名用户体验测试补充效果证据。上述创新属于面向银行业务场景的系统设计与工程集成创新，并非提出新的底层模型算法。

本文同时限定了研究范围。登录、注册和账户验证等功能是系统运行基础，不作为 AI 核心功能展开。论文重点分析源码中已经完成并可以演示的语音输入、

FAQ 知识库问答、异常情绪安抚、AI 查余额、AI 查流水、AI 账单统计与分析、AI 周报/月报和 AI 智能转账。至于消费品类识别、商户标签分析等当前源码没有实现的能力，本文不将其写入系统成果，只在不足与展望中作为后续方向讨论。

1.3 本文组织结构

全文结构保持九章安排。第一章说明选题背景、问题来源、研究目标和论文组织；第二章结合传统银行管理系统、行业 AI 实践和课题痛点，明确本文优化方向；第三章介绍大语言模型、Spring Boot、Vue、MyBatis、MySQL、ECharts 和 SSE 流式通信等技术基础；第四章分析系统总体需求、AI 核心功能需求和非功能性需求；第五章给出系统架构、AI 模块、数据库和接口交互设计；第六章依次说明语音输入、FAQ 知识库问答、情绪安抚、余额查询、流水查询、账单统计、周报/月报和智能转账的实现；第七章通过自动化测试、意图识别测试、功能测试、FAQ 检索统计、匿名用户体验测试和反馈分析验证系统；第八章总结主要结论、实践价值和不足展望；第九章作为结束语，对整个毕业设计过程进行收束。

第二章 相关研究与行业实践分析

2.1 传统银行管理系统与核心银行系统

银行管理系统本质上首先是一类高可靠业务信息系统。传统银行系统通常围绕核心银行系统建设，账户、客户、交易、存取款、贷款和财务会计等对象构成了主要业务流程。美国堪萨斯城联储关于核心银行系统现代化的研究指出，核心银行系统负责处理日常交易并更新账户和财务记录，基础服务包括账户管理、客户管理、存取款处理、贷款处理和财务会计，扩展服务还可能覆盖支付处理、客户支持、欺诈与风险管理、合规审计和报表分析等^[2]。因此，传统银行管理系统首先要保证流程规范、记录准确、账户状态一致以及后台服务稳定。

从软件工程实现看，这类系统最强调的是确定性。用户提交查询或交易请求后，系统需要依据登录身份、账户状态、交易规则和数据库记录给出结果。余额数值、流水是否存在、转账是否成功，都应来自数据库和后端服务，而不是来自语言模型的推断。本文的 BankCopilot 虽然只是本科毕业设计中的轻量级原型，但仍沿用这一原则：账户余额、交易流水、统计聚合和转账执行都由后端业务服务完成，AI 模块不直接生成业务事实。

不过，传统系统在人机交互上并不总是高效。菜单、表单和表格适合稳定的业务管理，却不一定适合用户快速提出复杂查询。例如用户想知道“这个月支出有没有异常”，通常还要自己选择时间范围、交易方向和统计方式，再阅读表格或图表。把智能客服接入传统银行管理系统后，系统可以把自然语言转为查询参数，再把聚合结果解释成更容易理解的说明。本文选择银行智能客服作为课题场景，正是基于这一交互改进空间。

2.2 银行 AI 应用的行业实践

金融科技和大语言模型的发展，使银行系统逐渐从传统信息管理工具向智能服务平台延伸。国际清算银行金融稳定研究所指出，人工智能会影响金融机构的运营效率、风险管理和客户体验，同时也引入模型风险、数据隐私、生成式 AI 幻觉、治理机制和第三方服务商管理等问题^[3]。因此，银行 AI 系统不能只看“能不能回答”，还要检查答案依据是否可靠、业务边界是否明确、敏感数据是否受保护、关键操作是否可追溯。

在客户服务领域，美国银行的虚拟金融助手 Erica 是较有代表性的案例。公开

资料显示, Erica 自 2018 年推出以来已服务近 5000 万用户, 累计完成超过 30 亿次客户交互, 并提供账户提醒、消费洞察和个性化财务建议等服务^[4]。该案例说明, 银行 AI 的价值之一, 是把用户从复杂页面路径中解放出来, 通过自然语言、主动提醒和个性化信息组织提升服务效率。

在业务流程自动化方面, JPMorgan Chase 的 COiN 合同智能平台展示了 AI 在信贷和合同处理中的应用价值。摩根大通年报披露, 该平台可以从 12000 份年度商业信贷协议中提取 150 个相关属性, 原本人工审查最多需要 36 万小时, 而系统可在数秒内完成^[5]。这类系统的意义不仅是节省人工时间, 还在于减少重复阅读和人工处理中的遗漏, 让复杂流程更容易自动化。

国内银行也在扩大大模型和智能化工具的应用范围。招商银行公开材料显示, 其已在零售服务、普惠金融授信、交易机器人、运营管理、风险管理和反洗钱等场景中使用大模型, 应用对象既包括客户, 也包括内部员工^[6]。由此可以看到, 银行 AI 已不再局限于客服问答, 而是逐步进入信贷审核、风险管理、运营支持和内部知识服务等综合场景。

这些行业实践给本文带来两点启发。第一, 银行 AI 的价值并不等同于聊天本身, 而在于它能否接入真实业务系统, 缩短查询路径、辅助业务判断并提升服务效率。第二, 银行 AI 必须配合治理边界, 在隐私保护、人工复核、解释说明和操作审计方面保持约束。本文系统规模远小于真实商业银行平台, 但在设计上借鉴了上述方向, 重点研究如何在轻量级 Web 银行业务原型中接入 AI 与传统业务服务。

2.3 银行 AI 系统面临的关键问题

银行场景中的智能化改造具有明显的两面性。一方面, AI 可以提升服务效率和交互体验; 另一方面, 银行业务涉及资金、身份和信用信息, 错误回答或错误操作可能带来较高风险。美国国家标准与技术研究院提出的 AI 风险管理框架强调, 可信 AI 应具备有效可靠、安全稳健、透明可问责、可解释、隐私增强和公平等特征^[7]。

在信贷和风险管理场景中, 上述问题更为明显。世界银行信用评分指南指出, 信用评分方法已从传统统计技术发展 to 机器学习、随机森林、梯度提升和深度神经网络等方法; 这些方法可以改进自动化程度、准确性和客户体验, 也可能引发隐私、公平性、可解释性和历史偏差等问题^[8]。

结合本文课题, 银行 AI 系统至少要处理五类问题。业务事实不能由模型临时生成, 余额、流水和转账结果要来自确定性业务系统; 高风险操作不能由模型直

接完成，转账等资金操作必须经过参数校验和用户确认；AI 回复应能说明依据，尤其是统计总结、异常提醒和趋势分析，需要明确时间范围、指标口径和数据来源；系统还要控制能力边界，不能回答源码中未实现的功能，也不能虚构消费品类、商户标签或授信结论；最后，智能交互要与原有页面共存，避免为了加入 AI 而破坏传统业务系统的可维护性。

因此，本文的研究问题可以概括为：在数据准确、业务可控、风险操作可确认的前提下，如何把自然语言交互、语音输入、统计解释和主动简报引入银行业务管理原型，让系统从“菜单表单式操作”扩展到“自然语言辅助操作”。这个问题不是单纯训练模型，也不是在页面上增加一个聊天框，而是要明确 AI 模块与传统后端业务服务之间的职责划分和交互边界。

2.4 本文系统与行业系统的差异

本文系统不能直接等同于真实商业银行的大规模 AI 平台。Bank of America、JPMorgan Chase 和招商银行等机构的 AI 系统一般依托完整核心银行系统、数据中台、风控平台、合规审计平台和长期运营数据，面对大规模用户、复杂组织流程和严格监管要求。本文的定位是本科毕业设计中的轻量级 Web 银行智能客服原型，目标不是复现大型银行平台，而是在可控范围内验证 AI 与传统业务管理系统结合的一种实现路径。

表 2.1 行业银行 AI 系统与本文系统对比

对比维度	行业系统特点	本文系统定位	本文优化重点
业务范围	覆盖账户、支付、信贷、风控、合规、运营等完整链路	面向个人账户业务的轻量级银行智能客服原型	聚焦余额、流水、统计、简报和转账确认等高频场景
客户交互	通过虚拟助手、移动银行和主动提醒提供个性化服务	在 Web 前端中提供文本与语音输入入口	将菜单式操作扩展为自然语言辅助操作
数据处理	依托真实生产数据、数据仓库和风控模型	使用项目数据库中的账户、收款人和交易流水数据	坚持由后端服务提供业务事实，AI 负责理解和表达
风险控制	包含反欺诈、反洗钱、信贷评分和合规审计等体系	主要完成转账校验、二次确认和能力边界控制	将高风险操作设计为“AI 解析 + 后端校验 + 用户确认”流程
AI 治理	强调模型评估、审计、隐私保护和人工复核	在原型系统中实现过程展示、统计口径说明和安全回退	使 AI 输出可解释、可检查，并避免超出已实现能力

从表 2.1 可以看出，本文系统的差异主要在于“小而精”的定位。系统不试图覆盖所有银行业务，而是选择几类具有代表性的 AI 集成场景：查余额体现确定性数据查询，查流水体现条件抽取和多轮补全，账单统计体现聚合数据解释，周报/月报体现主动信息组织，智能转账体现风险操作的分步确认，语音输入体现交互入口扩展。这些功能覆盖查询、统计、分析、简报和操作确认等典型模式，能够支撑“AI 如何嵌入传统银行管理系统”的讨论。

因此，本文的价值不在于声称系统已经达到商业银行平台水平，而在于借鉴行业 AI 应用思路，在本科毕业设计条件下完成一个边界清楚、实现完整、便于演示和验证的原型。借助该原型，可以观察大语言模型在银行业务系统中适合承担哪些工作，也可以看到哪些环节仍必须由传统后端服务和用户确认机制负责。

2.5 课题痛点与本文优化措施

针对传统页面路径较长、静态 FAQ 无法连接实时业务数据、大模型输出存在不确定性以及资金操作风险较高等问题，本文没有把智能客服做成独立闲聊模块，而是把它嵌入银行业务系统，作为自然语言辅助入口。具体优化措施如下。

第一，增加智能交互入口。传统 Web 银行业务系统主要依靠菜单、表单和表

格，用户需要自己寻找功能入口。本文在系统中加入 AI 助手面板，支持文本输入和语音转文字输入。用户可以直接说或输入“查询最近一周流水”“这个月支出了多少”“生成本周简报”等请求，系统再通过大模型解析意图，并由后端服务执行对应查询。语音输入不改变业务逻辑，只是把语音转成文本后复用原有 AI 请求链路，从而降低输入成本。

第二，明确大模型与后端服务的职责边界。本文不让大模型直接给出余额、流水或转账结果，而是把它限定为意图解析器和表达组织器。账户数据、流水数据、统计数据 and 资金变动都由 Spring Boot 后端服务从数据库查询或执行。这样可以减少模型幻觉对业务事实的影响，也能让每一次查询和操作回到具体服务方法和数据表。

第三，优化统计解释与主动简报能力。传统流水页面多以明细表格为主，用户需要自己理解收支变化。本文在账单统计和周报/月报中加入时间范围、指标口径、趋势数据、异常提醒和快捷追问，让系统不仅返回数据，还能解释数据。对用户来说，这减少了阅读表格和手动计算的成本；对论文研究来说，它体现了 AI 在信息组织和表达优化中的作用。

第四，优化风险操作流程。转账属于安全要求较高的银行操作。本文把智能转账拆成几个步骤：先由 AI 解析收款人、金额和备注，再由后端校验账户、余额、收款人和历史转账信息，最后由前端展示确认按钮，只有用户确认后才执行转账。该流程避免 AI 一次性完成资金操作，也保留了传统业务系统中必要的确认环节。

第五，增加可解释和过程展示机制。AI 回复中保留 trace、steps、explain、confidence、tool 和 status 等字段，前端可以展示意图、处理步骤和统计口径。用户看到的不只是最终回答，还能知道系统为什么这样回答、调用了哪类能力、统计范围是什么。对于银行智能客服，过程可见往往比单纯回复自然更重要，因为它能帮助用户判断结果是否可信。

第六，建立能力边界和安全回退机制。系统提示词明确限制模型不得输出未实现能力，消费品类、商户标签、复杂风控结论等源码中不存在的数据，不写成系统成果。同时，后端会对模型输出做结构化校验；当模型服务不可用或输出异常时，系统采用规则化后备回复。这样可以提高演示和测试时的稳定性，也更符合银行业务系统对可控性的要求。

第七，设置可验证的优化评价方式。本文不借用行业案例中的效率数据来证明本系统效果，而是通过功能测试和交互分析检查原型是否达到设计目标。评价内容包括自然语言请求是否正确解析，余额和流水是否与数据库一致，统计口径是否清楚，转账是否经过二次确认，FAQ 是否命中知识库，语音输入是否进入原

有文本链路，异常情绪是否触发安抚与人工客服引导，以及 AI 回复是否避开未实现能力。通过这些指标，可以更客观地说明优化措施是否有效。

表 2.2 优化措施与系统实现位置对应关系

优化措施	设计思路	具体实现章节	测试与评价章节
增加智能交互入口	将菜单式操作扩展为自然语言和语音转文字输入。	5.4、6.1、6.2	7.4 T1、T2、T11；7.5 U1、U2、U5
明确大模型与后端服务职责边界	大模型只做意图解析和表达组织，业务事实来自数据库和后端服务。	3.1、5.1、5.2、6.5–6.8	7.3、7.4、7.7
优化统计解释与主动简报	将流水明细扩展为总额、趋势、图表、简报和快捷追问。	5.4、6.7、6.8	7.4 T4–T6；7.5 U6
优化风险操作流程	转账采用“AI 解析 + 后端校验 + 用户确认 + 可取消”的流程。	4.3、6.9、表 6.5	7.4 T7–T9；7.5 U7
增加可解释和过程展示机制	回复中保留 <code>trace</code> 、 <code>steps</code> 、 <code>explain</code> 、 <code>chartData</code> 、 <code>exportUrl</code> 等字段。	5.2、6.1、6.7、附录 A.2	7.6 图 7.1、图 7.2
建立 FAQ 知识库与更新机制	常见问题从 <code>faq_knowledge</code> 表返回，检索结果写入 <code>faq_query_log</code> 。	5.3、6.4、附录 A.4、A.5	7.4 FAQ 检索统计；附录 A.6、A.7
建立能力边界和安全回退机制	对未实现能力进行拦截，避免模型编造消费分类、商户标签等结果。	3.2、4.3、6.1、6.7	7.4 T10；7.7
设置可验证评价方式	通过自动化测试、意图测试、功能测试、FAQ 统计、匿名测试日志验证系统效果。	5.3 <code>assistant_test_log</code> ；7.1–7.5；附录 A.7	7.2–7.5；附录 A.8–A.10

因此，第二章提出的优化措施会在后文形成对应闭环，而不是孤立陈述。第四章说明系统为什么需要这些约束，第五章回答如何设计，第六章回答如何实现，第七章则通过测试和反馈回答实现效果如何。

综合来看，本文创新点集中在四个方面。其一，在传统银行业务管理原型上加入文本与语音自然语言入口，使系统从表单式操作扩展为自然语言辅助操作，并补充 FAQ 知识库问答、异常情绪安抚和人工客服引导。其二，将大语言模型与确定性后端服务分工，AI 负责理解和表达，业务服务负责事实与执行。其三，围绕银行业务安全要求设计过程展示、统计解释、知识库来源、二次确认、取消回退和安全回退机制，使智能客服不仅能回答问题，也能以可解释、可控制的方式辅助完成业务。其四，增加 `assistant_test_log` 和匿名反馈测试入口，将功能验证、日志记录和用户反馈结合起来，使原型系统具备更完整的测试与反馈闭环。

第三章 相关技术与理论基础

3.1 大语言模型与自然语言意图识别

自然语言意图识别是智能客服进入业务流程前的第一步。系统需要从用户的口语化表达中判断其真正想完成的任务，并抽取时间、金额、收款人、交易方向等槽位信息。早期做法多依赖规则匹配、关键词识别或监督学习分类模型。Transformer 结构和大规模预训练语言模型的发展，使模型在长文本、上下文和少样本任务上的处理能力得到提升^[9,10]，因此更适合把用户输入整理成结构化意图。

本系统使用 QwenClient 统一封装通义千问兼容 OpenAI Chat Completions 的调用方式^[11]。在 AssistantService 中，parseIntentByQwen 方法把用户输入和系统提示词发送给模型，并要求返回严格 JSON。该 JSON 中包含 intent、toAccountNo、payeeAlias、amount、description、txnType、timeRange、metric、groupBy 等字段；后端再依据 intent 字段进入不同业务处理方法。

在银行业务场景中，大语言模型不能直接承担业务事实来源的角色。BankCopilot 只把模型用于意图解析和分析文本生成，余额、流水、统计值以及转账执行均由后端业务服务提供。这样既保留自然语言交互的灵活性，也降低了模型幻觉影响业务结果的风险。

对于银行智能客服，意图识别不是简单判断聊天主题，而是要把自然语言转换成后端接口可以使用的参数组合。例如，“这个月花了多少钱”应落到统计意图、支出方向、按月时间范围和求和指标；“给弟弟转 300 元”则应落到转账意图、收款人昵称和金额。模型输出越接近接口参数，后续链路越短，系统也越容易验证和维护。因此，本文要求模型输出结构化 JSON，并在服务层继续校验。

大语言模型还用于生成分析性表达。对于统计结果，后端已经计算出总额、趋势和对比数据，若只返回数字，用户不一定能快速理解。因此，系统会先整理统计事实，再让模型生成客服式说明。这里仍需限制模型边界：分析文本只能基于后端事实数据，不能扩展出数据库中不存在的交易类型、商户类别或风险结论。

3.2 自然语言处理方案选择与比较

任务书要求课题对自然语言处理和机器学习方案进行研究与选择。结合银行智能客服场景，本文在设计阶段比较了关键词规则匹配、传统文本分类模型、检索式 FAQ 问答、大语言模型意图解析和情感分析等方案。由于系统目标不是开放

式闲聊，而是把用户请求转换为后端可执行的银行业务意图，方案选择时更关注意图识别、槽位抽取、扩展成本和业务可控性。

表 3.1 自然语言处理方案比较

方案	优点	不足	本文采用情况
关键词规则匹配	实现简单、结果可控、便于定位问题	对口语化表达、同义表达和复杂句式适应性较弱，规则数量增加后维护成本较高	用于不支持能力拦截和部分后备判断
传统文本分类模型	可通过训练集计算准确率、召回率等指标，适合固定意图分类	需要标注语料，新增意图或槽位时需要重新训练或补充样本	未作为主方案
检索式 FAQ 问答	适合常见问题和制度说明，答案稳定	难以直接完成余额、流水、统计和转账等实时业务查询	作为常见咨询补充方案
大语言模型意图解析	对口语表达、复杂句式和多槽位抽取适应性较强，可直接输出结构化 JSON	存在格式不稳定、幻觉和超出能力边界表达的风险	作为本文主方案
情感分析	可用于识别明显情绪倾向，辅助人工客服引导或投诉识别	若作为复杂情绪分类模型，需要更多标注语料；但投诉、愤怒、焦虑等强表达可通过规则识别	未训练复杂情感分析模型，仅采用轻量级关键词规则检测

从表 3.1 可以看出，传统文本分类模型便于做标准机器学习评估，但前提是有足够的标注语料。本课题属于本科毕业设计原型，意图类别并不多，但每类意图都带有多个槽位。比如查流水既要识别 `transactions`，也要抽取收入或支出、时间范围、查询条数以及是否需要补全月份；智能转账既要识别 `transfer`，也要抽取收款人、金额、备注和历史金额复用方式。若只采用普通文本分类模型，还要额外编写槽位抽取规则，整体开发成本反而较高。

检索式 FAQ 适合处理“如何开卡”“密码忘记怎么办”等常见咨询，但它不能替代实时业务服务。余额、流水和统计结果必须来自数据库，转账也必须经过后端校验和用户确认，单纯检索固定答案无法满足这些场景。因此，本文把 FAQ 检索定位为常见咨询补充模块：当业务意图为 `other` 时，系统才从数据库知识库表中检索标准答案或相似问题。

综合比较后，本文采用“大语言模型意图解析 + 后端确定性服务 + FAQ 检索 + 规则化后备处理 + 情绪关键词检测”的混合方案。`AssistantService` 中的 `parseIntentByQwen` 要求通义千问输出严格 JSON，将用户输入拆解为 `intent`、

txnType、timeRange、amount、payeeAlias、metric、groupBy 等字段；后端再根据 intent 进入余额、流水、统计、转账或 FAQ 知识库流程。对于消费分类、商户类型、场景标签等当前系统不支持的能力，后端通过规则拦截并转入 other 分支，避免模型把未实现能力描述成已实现成果。

本文未单独训练情感分类模型，而是针对客服场景实现了轻量级异常情绪关键词检测。当规则未命中且请求进入后备回答分支时，系统仍会尽量生成温和回复。具体实现中，“我要投诉”“服务太差”“生气”“焦虑”“担心”等较强情绪或投诉表达会触发安抚话术和人工客服引导。这样设计的原因是，当前系统核心业务仍是账户查询、流水查询、统计分析和转账确认，情绪判断不应参与金额计算或资金操作决策；但在客服交互中，识别明显负面情绪并提示人工渠道可以补全服务体验。因此，本文把该能力定位为交互优化辅助模块，而不是复杂情感分析模型或核心业务决策模型。

3.3 Spring Boot 后端技术

Spring Boot 是本系统后端的主体框架，用于构建 RESTful Web 服务^[12]。项目后端按照 Controller、Service、Mapper 分层组织代码。AssistantController 负责暴露 AI 相关接口，AssistantService 负责请求编排，AccountService、TransactionService、StatisticsService 和 PayeeService 分别提供账户、流水、统计和收款人相关能力。

后端还使用 Spring MVC 拦截器完成 Token 校验，通过 TokenInterceptor 解析 JWT，并把当前用户 id 保存到 CurrentHolder。AI 接口在调用业务服务前会读取当前用户 id，保证查询和操作都发生在登录用户上下文中。登录鉴权并不是本文重点，但它是 AI 查询个人账户数据和执行转账操作的前置条件。

从稳定性角度看，Spring Boot 后端承担业务可信边界。模型服务可能受到网络、限流或返回格式异常影响，但后端必须保证接口输入、权限边界、金额计算和数据库更新的确定性。AssistantService 在调用模型后，只根据解析结果进入固定分支，再调用对应业务服务。这样可以把模型能力限制在服务编排层，避免其绕过后端既有校验逻辑。

3.4 Vue 与前端交互技术

前端使用 Vue 3 构建组件化界面^[13]，并通过 Element Plus 提供表单、按钮、抽屉、弹窗和表格等基础组件。AI 助手界面主要由 AssistantPanel.vue 实现，该组件既用于首页主面板，也用于右上角智能客服抽屉。

在 AI 交互中，前端通过 `fetch` 调用 `/assistant/chat/stream`，读取服务端发送事件流，并根据 `trace`、`message`、`error`、`done` 等事件更新界面。统计分析结果则根据 `chartType` 和 `chartData` 交给 ECharts 渲染为柱状图、折线图、饼图或概览图^[14]。当回复处于转账确认状态时，前端会显示“确认”和“取消”按钮，减少用户再次手动输入。

输入方式方面，AssistantPanel 使用浏览器提供的 `SpeechRecognition` 或 `webkit-SpeechRecognition` 实现中文语音识别。该功能没有单独训练语音模型，而是调用浏览器语音接口把语音转写为文本，再填入聊天输入框。后续仍沿用原有 AI 文本请求链路，因此语音输入本质上是自然语言入口的扩展，而不是新的后端业务流程。

前端交互会直接影响智能客服的可用性。如果系统只在页面底部显示一段长文本，用户很难判断 AI 是否真正调用了业务服务，也难以看清统计结果口径。因此，AssistantPanel 在展示回复文本之外，还展示处理过程、图表、快捷按钮和导出口。对于答辩展示，这些可视化元素也能让系统能力呈现在页面上，而不是只停留在控制台或接口返回层面。

3.5 数据持久化与统计分析技术

系统使用 MySQL 保存用户、账户、收款人和交易流水数据。MyBatis 负责 SQL 映射^[15]，其中 `TransactionMapper.xml` 定义了流水分页查询、总金额统计、笔数统计、最大交易查询、趋势聚合、收入支出对比、按对方账号聚合以及历史转账流水匹配等 SQL。

统计分析由 `StatisticsServiceImpl` 负责，它根据前端或 AI 解析出的 `StatisticsQueryDTO` 选择 `sum`、`count`、`max`、`trend` 等统计方式。由于支出流水在数据库中以负数保存，SQL 中使用 `ABS(amount)` 转为正向统计值，便于用户阅读。统计结果随后由 `AssistantService` 组织成事实文本，再交给大语言模型生成分析总结；若模型不可用，则使用规则化后备总结。

数据持久化层为多项 AI 功能提供事实依据：余额查询依赖账户表，流水查询和统计分析依赖交易流水表，智能转账依赖常用收款人表和账户表，历史金额复用依赖历史交易记录。虽然这些功能在前端都表现为聊天输入，但后端的数据读取和更新路径并不相同。后续设计与实现章节会分别说明这些路径，以体现系统不是简单拼接模型接口，而是围绕银行业务数据建立完整功能链路。

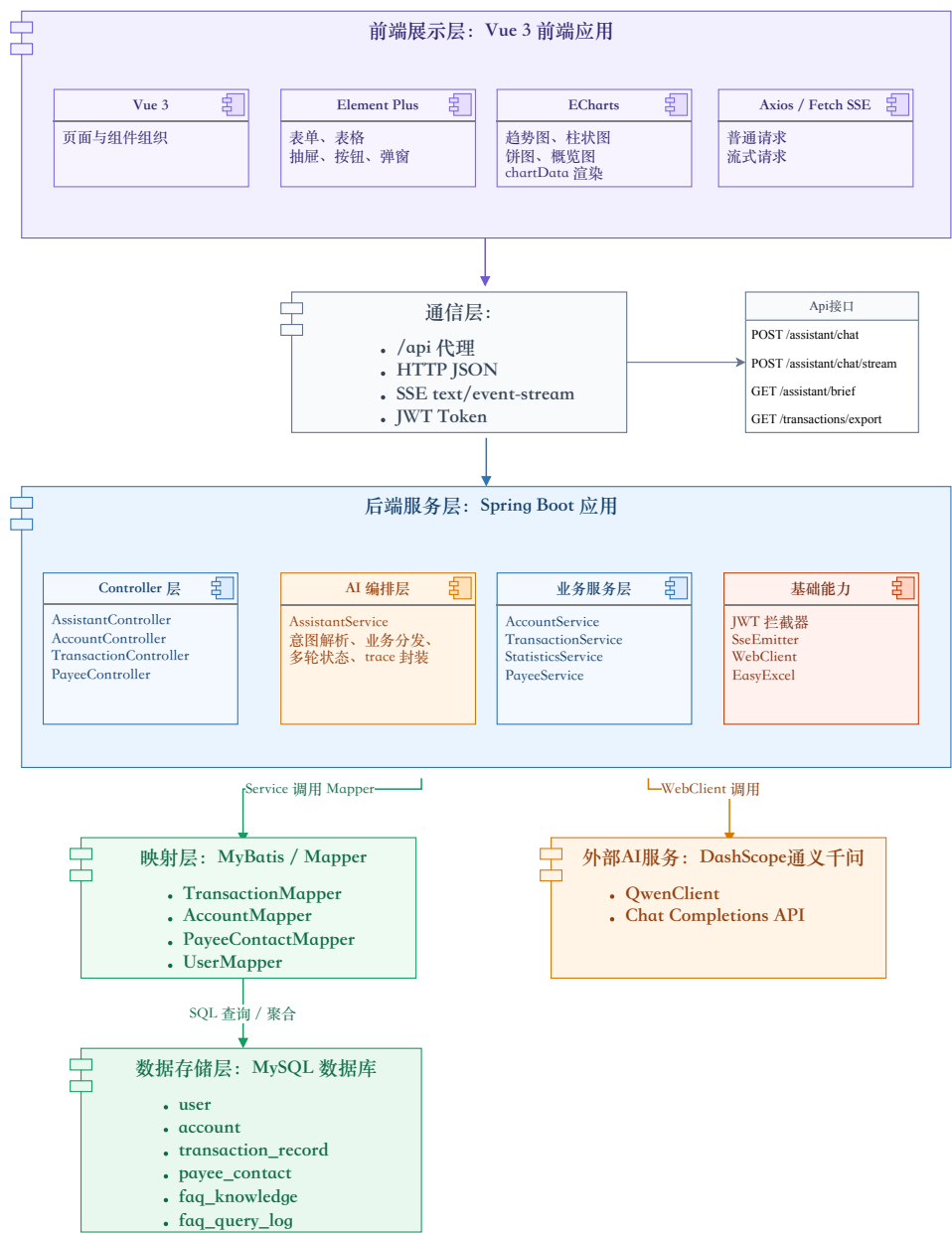


图 3.1 系统技术栈结构图

第四章 系统需求分析

4.1 系统总体需求

BankCopilot 面向个人银行用户，目标是降低常见银行业务的操作复杂度。用户不必记忆具体菜单和筛选条件，可以直接用自然语言表达需求。系统需要识别用户意图，把它映射到后端业务服务，再以文本、图表或导出文件等形式返回结果。

从用户角度看，系统应支持账户余额查询、交易流水查询、收支统计分析、周报/月报查看和转账确认等能力。从系统实现角度看，AI 助手则需要具备意图识别、槽位抽取、时间解析、多轮状态管理、业务服务调用、结果格式化和前端可视化展示等能力。

总体需求还体现在边界划分上。AI 助手可以提供更自然的入口，但不能替代后端业务规则。用户表达越口语化，后端越需要明确转换和校验逻辑。例如，用户说“这个月花了多少”时，系统必须确定起止时间、交易方向和统计指标；用户说“给弟弟转 300”时，系统必须识别收款人、校验金额、检查余额并等待确认。自然语言表达只有与确定性业务规则结合，系统才具备银行场景下的可用性。

此外，考虑到本科毕业设计展示和答辩评阅，系统页面不能只呈现 AI 回复文本，还应展示调用了什么业务能力、依据了哪些数据、是否触发安全确认。因此，本系统在需求阶段就把处理过程展示、统计图表展示和快捷操作按钮纳入设计。这些元素不是页面装饰，而是证明系统存在真实业务链路的重要交互证据。

4.2 AI 核心功能需求

表 4.1 AI 核心功能需求表

功能编号	功能名称	需求描述
F1	AI 查余额	用户通过自然语言查询账户余额和账户基本信息，系统返回户名、账号、账户类型、状态和余额。
F2	AI 查流水	用户通过自然语言查询交易记录，系统支持收入/支出筛选、时间范围和查询条数解析。
F3	AI 账单统计与分析	用户查询总金额、笔数、最大交易或趋势时，系统基于数据库统计结果生成文本分析和图表数据。
F4	AI 周报/月报	用户打开 AI 面板时，系统展示本周或本月收入、支出、净流入、趋势、对比和异常提醒。
F5	AI 智能转账	用户通过自然语言发起转账，系统解析收款人、金额和备注，校验后等待用户确认再执行。
F6	AI 语音输入	用户点击语音输入按钮后，系统识别中文语音并回填到聊天输入框，后续按普通文本请求继续处理。
F7	FAQ 知识库问答	用户咨询银行卡挂失、转账到账时间、密码找回、验证码、贷款材料等常见问题时，系统从数据库知识库中检索并返回标准答案。
F8	AI 情绪安抚与人工客服引导	当用户表达投诉、焦虑、愤怒或担心等异常情绪时，系统先进行简短安抚，并提示拨打虚拟客服电话联系人工客服。

除表中功能外，系统还需要若干 AI 基础能力，包括自然语言意图识别、时间范围纠偏、SSE 流式响应、AI 处理过程展示、统计口径解释和 Excel 导出链接生成等。它们本身不一定是用户直接提出的业务目标，却会影响交互体验和系统可解释性。

4.3 非功能性需求

安全性方面，系统要保证用户只能查询自己的账户和流水数据，资金流动操作必须经过校验和二次确认。AI 转账不能在用户首次输入后直接扣款，而是先生成确认信息，待用户回复“确认”后再执行。聊天回复中的对方账号应进行脱敏展示。

准确性方面，账户余额、流水记录和统计金额都必须来自数据库查询结果，不能由大模型直接生成。模型输出的统计总结要受到后端事实约束；当模型返回

内容不合适时，系统需要切换到规则化后备总结。

可用性方面，系统应支持“查一下我的余额”“看最近 7 天支出趋势”“给弟弟转 300 元”等口语化输入。对于缺少关键信息的请求，系统应通过多轮对话引导用户补全，而不是直接报错结束。

交互性方面，AI 助手应减少用户等待感和理解成本。系统通过流式响应展示处理过程，通过图表呈现统计结果，通过快捷按钮完成转账确认或简报追问，以提高交互效率。

可维护性方面，系统应保持前后端职责清晰。前端负责交互展示和输入收集，后端负责业务判断和数据访问，大模型调用封装在独立客户端中。采用这种划分后，后续若需要更换模型、调整提示词或增加统计指标，可以优先修改 QwenClient、AssistantService 或 StatisticsService，而不必大范围改动前端页面和数据库结构。清晰的模块边界有利于后续维护和迭代。

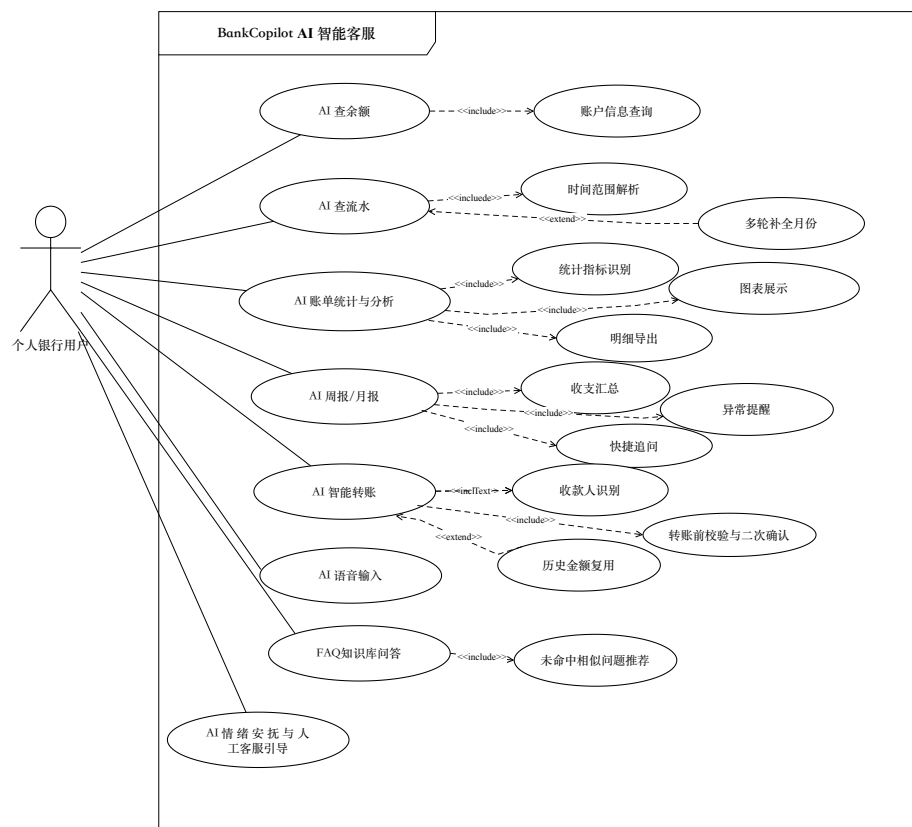


图 4.1 AI 智能客服用例图

第五章 系统总体设计

5.1 系统架构设计

系统采用前后端分离架构。Vue 3 应用负责页面展示和用户交互，Spring Boot 应用负责业务处理和数据访问，MySQL 负责数据持久化，DashScope 通义千问用于自然语言意图解析和分析性回复生成。前端通过 /api 代理访问后端接口，后端通过 WebClient 调用大模型服务。系统总体架构如图5.1 所示。

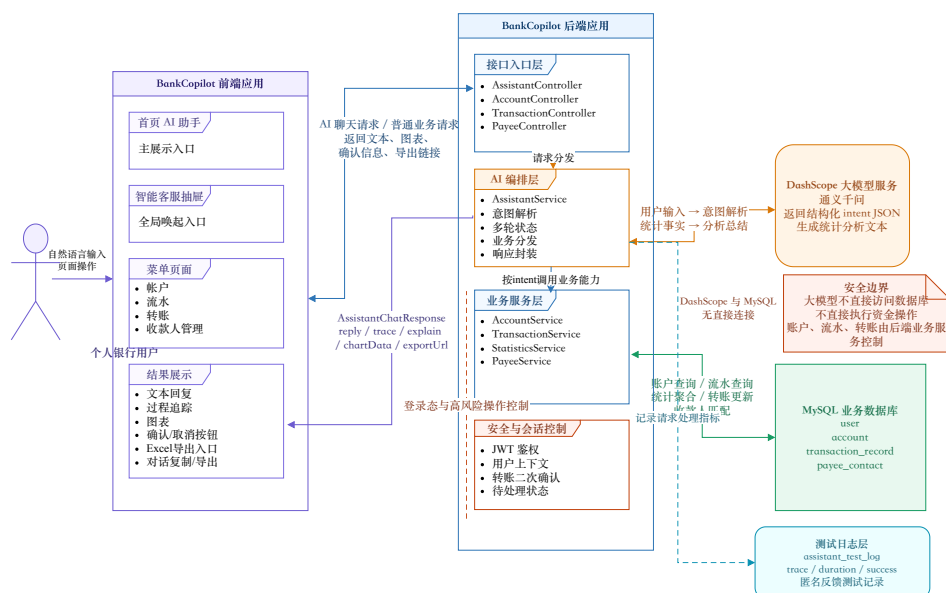


图 5.1 系统总体架构图

在该架构中，AI 助手不是业务系统之外的独立聊天模块，而是位于业务系统上层的自然语言交互入口。用户在前端智能助手输入请求后，请求先进入后端接口层，再进入 AI 编排层；系统通过 DashScope 解析业务意图，并按 intent 字段分发到账户、流水、统计或收款人服务。相关服务查询或更新 MySQL 后，再把结果封装返回前端。需要强调的是，大模型不能直接访问数据库，也不执行资金操作；账户、流水和转账均由后端业务服务控制。因此，架构层面的核心设计并不是“在页面上增加聊天窗口”，而是把 AI 助手嵌入传统业务链路：前端收集自然语言输入并展示处理过程，AI 编排层把输入转换为可执行任务，业务服务层提供事实数据和执行结果。通过这种分层，系统既保留传统银行业务系统的确定性，又为高频查询和咨询提供更短的自然语言入口。

5.2 AI 模块设计

AI 模块按职责拆分为接口层、编排层、模型调用层、业务服务层、知识库服务层和前端展示层。接口层对应 `AssistantController`，提供 `/assistant/chat`、`/assistant/chat/stream` 和 `/assistant/brief` 三类接口；编排层对应 `AssistantService`，负责待确认状态处理、`parseIntentByQwen` 调用、`intent` 分发以及 `trace` 和 `explain` 封装；模型调用层由 `QwenClient` 实现，统一封装通义千问请求；业务服务层包括 `AccountService`、`TransactionService`、`StatisticsService` 和 `PayeeService`；知识库服务层由 `FaqKnowledgeService` 和 `FaqKnowledgeMapper` 负责常见问题检索和查询日志记录；前端展示层由 `AssistantPanel` 完成，并包含语音输入按钮与识别状态展示。

这种模块划分的核心思想是“模型不直接接触数据库，业务不依赖模型自由发挥”。如图5.2所示，`AssistantService` 位于 AI 模块中心，它既要读取模型返回的意图字段，也要根据业务规则判断能否继续执行。普通查询可以直接调用账户或流水服务；缺少月份的流水查询要先保存待补全状态并追问用户；转账请求则要生成待确认对象，等待用户下一轮确认。不同路径都从聊天输入开始，但后端处理策略并不相同。

AI 模块还负责结果适配。业务服务返回的通常是对象、列表或聚合数据，而用户更容易理解简明文本、图表和下一步操作入口。因此，`AssistantService` 会把业务结果转换为 `AssistantChatResponse`，前端再根据 `chartType`、`chartData`、`trace`、`explain` 和 `exportUrl` 等字段决定展示方式。这样可以把回复内容与展示形式解耦，后续增加图表类型或统计指标时，不必破坏原有接口。由此可见，AI 模块承担的是业务入口、服务编排和表达适配三类职责，而不是替代账户、流水、统计和转账服务本身。这个职责边界是本文系统设计的基础，也是后续安全控制、可解释展示和测试验证能够成立的前提。

表 5.1 AI 模块主要职责

模块	源码位置	主要职责
AI 接口入口	AssistantController.java	接收聊天请求、流式聊天请求和简报请求。
AI 编排服务	AssistantService.java	意图解析、业务分发、多轮状态、FAQ 后备处理、统计总结。
模型调用	QwenClient.java	调用通义千问 Chat Completions 接口。
知识库检索	FaqKnowledgeService.java	检索 FAQ 知识库，返回直接命中、相似推荐或未命中结果。
时间处理	Time.java	自然语言时间纠偏和时间范围填充。
前端面板	AssistantPanel.vue	聊天、语音输入、简报、图表、过程追踪和确认按钮展示。

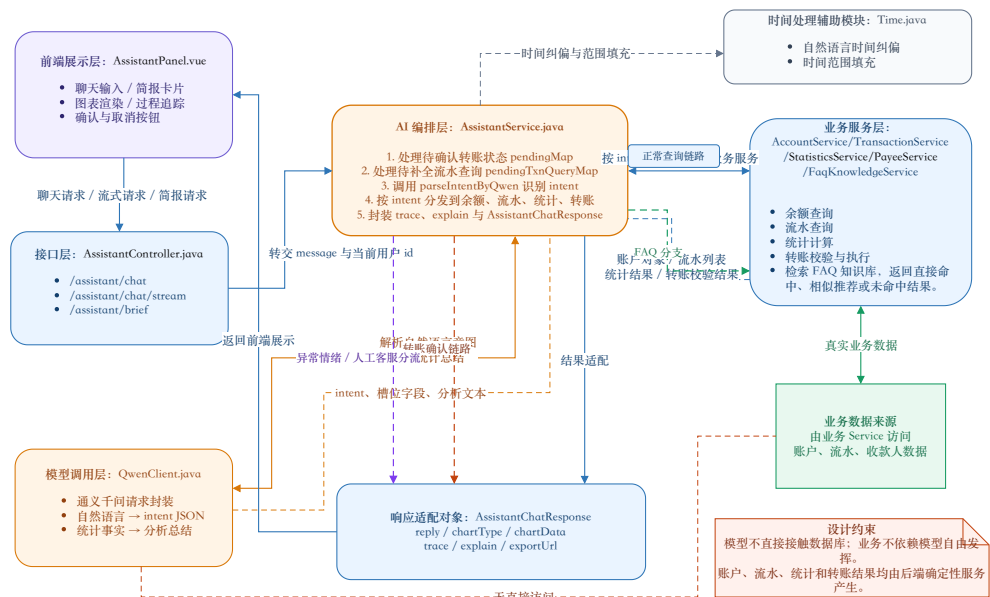


图 5.2 AI 请求处理架构图

5.3 数据库设计

系统 AI 功能依赖的核心数据表包括 `account`、`transaction_record`、`payee_contact`、`user`、`faq_knowledge` 和 `faq_query_log`。AI 查余额主要读取 `account` 与 `user` 表；AI 查流水和账单统计主要读取 `transaction_record` 表；AI 智能转账需要读取 `account` 和 `payee_contact` 表，并在执行后写入 `transaction_record` 表；FAQ 知识库问答读取 `faq_knowledge` 表，并把检索结果写入 `faq_query_log` 表，便于后续统计命中情况。为补充测试数据来源，系统还设计 `assistant_test_log` 表，用于记录 AI 助手请求耗时、结果状态、意图、工具调用和回复长度等指标。

表 5.2 AI 功能相关数据表说明

数据表	用途
user	保存用户基本信息，为账户户名和登录用户上下文提供支持。
account	保存银行账号、账户类型、余额和状态，是查余额和转账的基础表。
transaction_record	保存收入、支出、金额、对方账号、备注和交易时间，是流水查询和统计分析的数据来源。
payee_contact	保存用户收款人、银行卡号和昵称，用于 AI 按昵称转账。
faq_knowledge	保存 FAQ 分类、标准问题、相似问法、关键词、标准答案和启用状态，用于常见问题知识库检索。
faq_query_log	保存用户原始问题、命中 FAQ、匹配得分和结果类型，用于统计知识库命中率和未命中问题。
assistant_test_log	保存 AI 助手测试和运行指标，包括请求来源、场景、意图、调用工具、处理状态、耗时和结果状态，用于补充第七章测试数据来源。

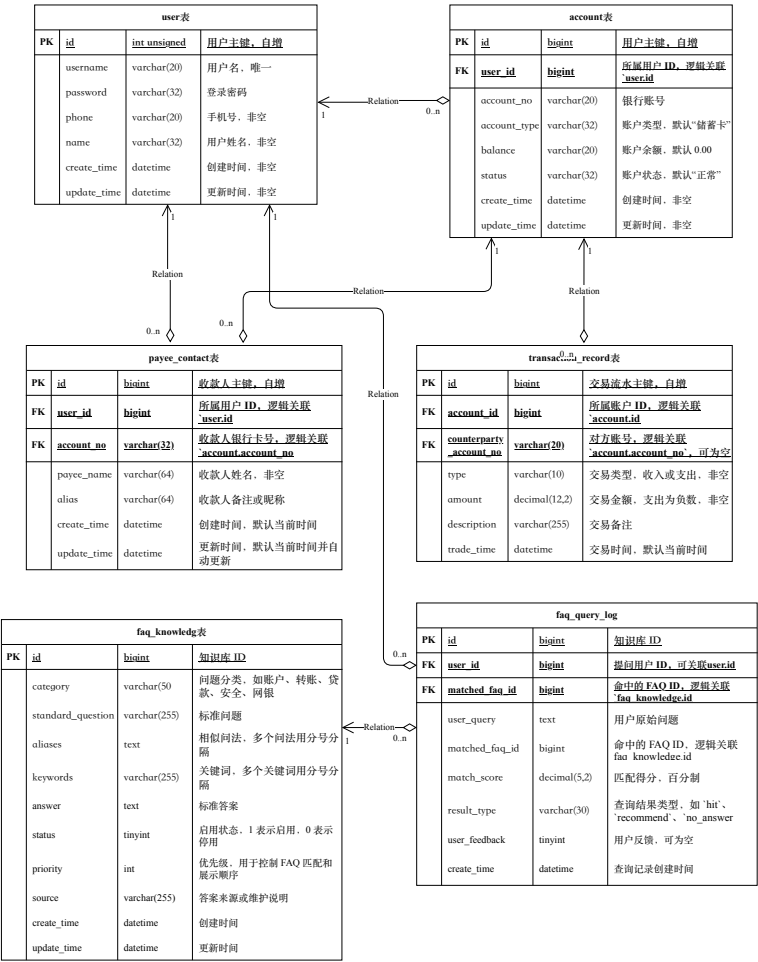


图 5.3 数据库 ER 图

5.4 接口与交互设计

AI 相关接口主要包括普通聊天接口、流式聊天接口、简报接口和 Excel 导出接口。普通聊天接口用于非流式请求或流式失败后的降级处理；流式聊天接口用于降低用户等待感；简报接口用于获取周报或月报数据；导出接口用于导出明细数据 Excel。

表 5.3 AI 相关接口设计表

接口	方法	功能
/assistant/chat	POST	普通 AI 聊天请求，返回完整 AssistantChatResponse。
/assistant/chat/stream	POST	SSE 流式 AI 聊天请求，返回 trace、message、done 等事件。
/assistant/brief	GET	获取 AI 周报或月报简报。
/transactions/export	GET	根据统计口径导出交易流水 Excel。

前端交互设计上，AI 面板既可以作为首页主体展示，也可以通过右上角智能客服抽屉打开。面板顶部展示周报/月报卡片，中间展示聊天记录和图表，底部提供文本输入框、演示脚本按钮、语音输入按钮和发送按钮。当 AI 回复处于转账待确认状态时，前端会显示确认和取消快捷按钮。

接口设计遵循前后端分离项目的基本原则。普通聊天接口适合一次性返回结果，便于调试和后备处理；流式聊天接口面向实际交互，可以让前端先显示处理步骤，再显示最终回复；简报接口用于页面初始化，用户打开 AI 面板即可看到近期账户概览。语音识别不新增后端接口，识别结果直接回填原有聊天输入框。不同接口对应不同交互时机，避免所有逻辑集中到单一接口中，也便于后续测试和维护。

第六章 AI 智能客服核心功能设计与实现

本章对应第一章和第二章提出的创新点与优化措施，重点说明它们在源码中的落地方式。6.1 介绍 AI 请求处理主流程和多轮状态优先机制，对应自然语言业务入口、对话管理和过程展示；6.2 介绍语音转文字输入，对应交互入口扩展；6.3 和 6.4 分别说明异常情绪安抚、人工客服引导和 FAQ 知识库问答，对应客服分流和知识库建设；6.5 至 6.8 说明余额、流水、统计和周报/月报，对应模型与后端确定性服务分工、统计解释和主动简报；6.9 说明智能转账二次确认、历史金额复用和取消回退，对应风险操作控制。通过这些实现，前文提出的优化措施被落实为可演示、可测试的系统功能。

6.1 AI 请求处理总体流程

`AssistantService` 的 `chat` 方法是 AI 请求处理入口。方法开始时先校验用户输入是否为空，随后优先检查待确认转账和待补全流水查询。如果当前用户存在未完成的多轮状态，系统会把本轮输入用于继续该状态；若不存在，则先用规则检测明显负面情绪或投诉表达，再调用 `parseIntentByQwen` 解析新请求。解析完成后，系统按照 `intent` 字段分发到 `handleTransfer`、`handleTransactions`、`handleBalance`、`handleStatistics` 或 `handleFaqOrFallback`。其中 `handleFaqOrFallback` 会先检索 FAQ 知识库，无法命中时再进入后备回复。

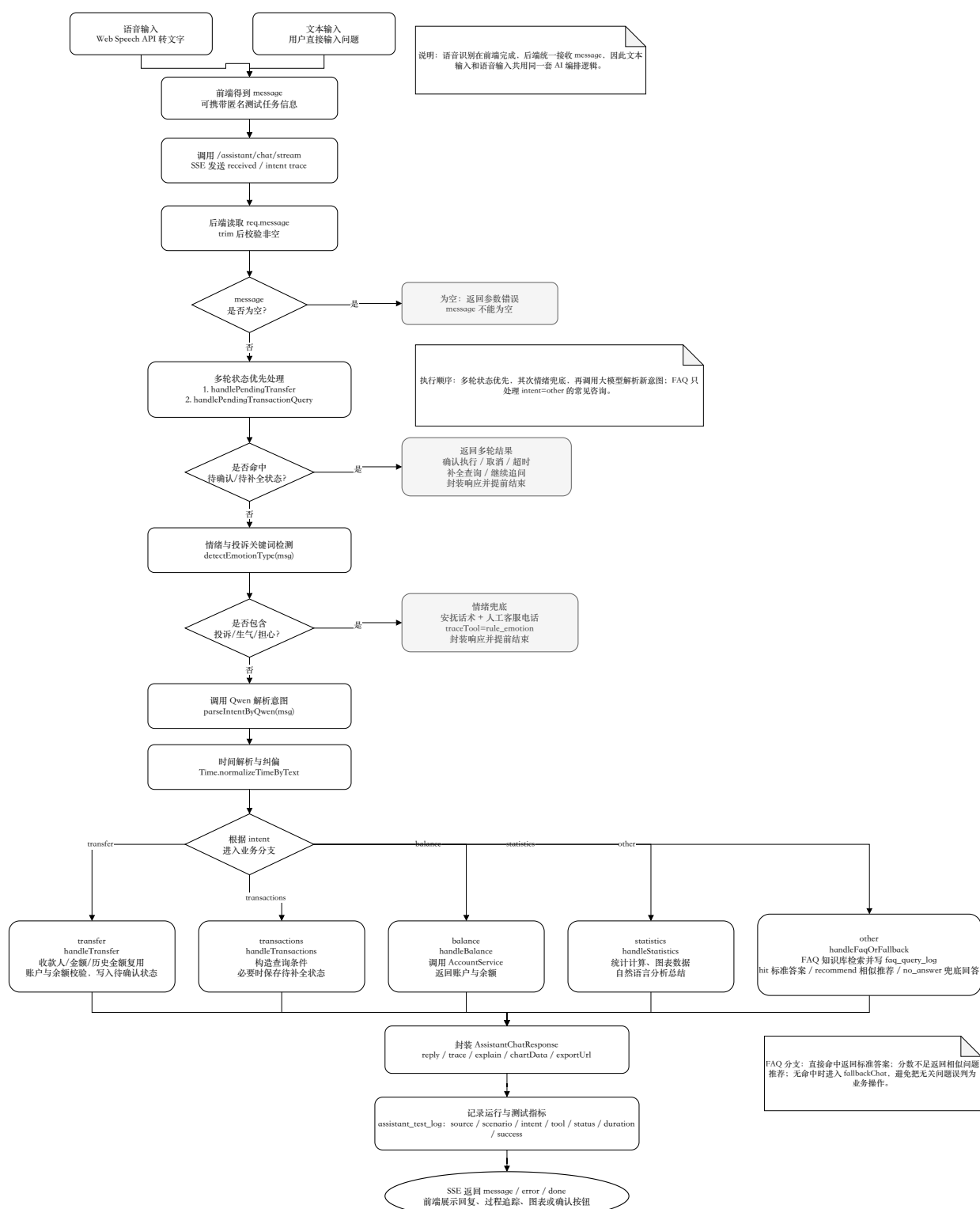


图 6.1 AI 聊天请求总体流程图

该流程体现了“多轮状态优先，新请求解析其次”的处理顺序。用户发起转账后，系统等待“确认”或“取消”，此时下一条输入应优先作为转账确认处理，而不是重新解析为普通聊天请求。类似地，当用户查询“17 号收入记录”却缺少

月份时，系统会先保存待补全状态，下一条“1月”应被视为补充条件。

这种处理顺序符合聊天式业务系统的实现规律。单轮问答只需要根据当前输入返回结果，而银行客服机器人需要理解前后两轮输入之间的关系。若不维护状态，系统无法判断“确认”对应哪一笔转账，也无法判断“1月”是在补充哪一次查询。

响应结构上，系统不只返回 `reply`，还返回 `trace`、`chartData`、`exportUrl` 和 `explain` 等字段。`reply` 面向普通用户，强调可读性；`trace` 用于解释和调试，记录识别意图、调用工具和处理步骤；`chartData` 供前端图表展示；`exportUrl` 用于交易流水导出；`explain` 用于说明统计口径。借助这些结构化字段，前端可以把一次 AI 回复拆成文本、图表、过程和操作入口，使系统更接近完整的智能业务助手，而不是简单聊天窗口。

6.2 AI 语音输入功能实现

语音输入功能主要由前端实现，通过浏览器语音识别能力完成中文语音转写。系统面向中文短句输入，在识别过程中可以把临时结果实时显示到输入框中。

用户点击“语音输入”按钮后，`toggleVoiceInput` 方法启动语音识别，并将 `voiceListening` 设为 `true`。识别过程中，`onresult` 事件读取 `event.results` 中的转写文本，再通过 `mergeSpeechText` 与输入框原有内容合并，最终写入 `chatInput`。用户可以在识别后手动修改，也可以直接点击发送。后续流程与普通文本输入一致，仍通过 `/assistant/chat/stream` 发送到后端，再由 `AssistantService` 完成意图解析、业务分发和结果封装。

该功能主要用于满足语音识别交互要求，并降低移动端或演示场景下的输入成本。与键盘输入相比，语音识别可能出现语句不通顺、字词遗漏、转写错误或表达冗长等情况，这也能用于观察 AI 在接近真实交流场景下的处理效果。由于语音识别只负责把语音转成文本，不直接调用账户、流水、统计或转账服务，因此不会改变原有业务安全边界。当浏览器不支持语音识别、麦克风未授权或未识别到有效内容时，前端会给出提示，用户仍可回到文本输入方式完成同一业务操作。

6.3 AI 情绪安抚与人工客服引导功能实现

情绪安抚与人工客服引导主要在后端 `AssistantService` 中实现。系统在处理转账确认和流水查询补全之后、调用大模型解析普通意图之前，先调用 `handleEmotionSupport` 对用户输入做轻量级情绪检测。该检测不依赖单独训练的情

感分类模型，而是通过关键词规则识别投诉、愤怒、焦虑和担心等明显表达。例如，输入包含“我要投诉”“投诉你们”“服务太差”“乱扣”“扣错”等词时归为 **complaint**；包含“生气”“气死”“愤怒”“崩溃”等词时归为 **angry**；包含“很着急”“焦虑”“担心”“害怕”等词时归为 **anxious**。

检测到异常情绪后，系统不会继续让大模型自由生成安抚内容，而是返回规则化回复。回复包括三部分：先对用户情绪进行简短理解和安抚；再提醒涉及账户和资金问题时不要重复提交操作，并说明系统仍可继续协助查询余额、流水、统计或转账状态；最后提示用户拨打虚拟客服电话 010-1234-5678 联系人工客服。该号码仅用于毕业设计演示，不对应真实银行热线。

同时，关键词规则无法覆盖所有表达，因此当请求进入后备回答分支时，系统仍会尽量生成较温和的安抚性回复。

该功能的边界需要说明清楚。情绪安抚模块不参与账户余额计算、流水查询、统计聚合和转账执行，也不会改变资金操作结果。它只在用户出现明显负面情绪或投诉意图时，提供更符合客服场景的回应，并把可能需要人工处理的问题引导到人工客服渠道。这样既回应了任务书中关于情感分析类能力和交互优化的要求，也避免把关键词检测夸大为复杂情感分析模型，或误用为银行业务决策依据。

6.4 FAQ 知识库问答功能实现

FAQ 知识库问答用于处理不属于余额、流水、统计和转账执行的常见咨询。系统新增 **faq_knowledge** 表，保存问题分类、标准问题、相似问法、关键词、标准答案、启用状态和优先级等字段；同时新增 **faq_query_log** 表，记录用户原始问题、命中 FAQ、匹配得分和结果类型。这样既满足“建立知识存储常见问题和答案”的要求，也为后续测试统计提供数据来源。

处理流程上，**AssistantService** 完成业务意图分发后，如果 **intent** 为 **other**，就进入 **handleFaqOrFallback**。该方法先调用 **FaqKnowledgeService.search**，将用户问题与知识库中的标准问题、相似问法和关键词进行匹配。匹配得分达到高置信度阈值时，系统直接返回标准答案；得分处于中等区间时，系统返回 2 到 3 个相似问题供用户继续选择；知识库未命中时，再进入 **fallbackChat** 后备回复。

该设计可以避免大模型自由编写银行业务规则。银行卡挂失、转账到账时间、验证码收不到、密码找回、贷款材料等问题的答案都来自 **faq_knowledge** 表，而不是模型临时生成。**faq_query_log** 表还可以统计 **hit**、**recommend** 和 **no_answer** 三类结果，用于第七章分析知识库直接命中、相似推荐和未收录问题情况。

知识库更新方面，当前系统通过数据库表维护 FAQ 内容。**faq_knowledge** 表

中的 status、priority 和 update_time 字段用于控制答案启用状态、返回优先级和更新时间。当用户问题没有直接命中知识库时，系统会把原始问题、匹配结果和 result_type 写入 faq_query_log 表。后续维护时，管理人员可以根据 no_answer 和 recommend 记录补充标准问题、相似问法、关键词和标准答案，并在审核后启用。通过“查询日志反馈 + 人工维护审核 + 数据库字段控制”的方式，系统在原型范围内回应了知识库持续更新的要求。

6.5 AI 查余额功能实现

AI 查余额对应 balance 意图。用户输入“查一下我的余额”“我的账户信息”等表达时，parseIntentByQwen 会把 intent 解析为 balance。随后 AssistantService 调用 handleBalance，并通过 AccountServiceImpl.getAccountInfo 查询当前登录用户的账户信息。

handleBalance 返回户名、账号、账户类型、账户状态和余额。该功能逻辑相对直接，但体现了系统设计中的关键原则：AI 只负责识别用户想查余额，余额数值本身必须来自数据库账户表，而不是由模型生成。

余额查询虽然业务逻辑不复杂，但在本文中具有基础意义。它证明 AI 助手能够从自然语言入口进入真实后端服务，而不是只返回固定话术。同时，余额查询也为后续流水查询、统计分析和转账功能提供账户上下文。若余额查询无法保证准确，后续涉及金额分析和资金操作的功能就缺少可信前提，因此本文将其作为核心功能之一说明。

表 6.1 AI 查余额功能实现说明

输入示例	解析意图	调用方法	返回内容
查一下我的余额	balance	handleBalance	户名、账号、账户类型、状态、余额。
我的账户信息	balance	handleBalance	账户基本信息和余额。

6.6 AI 查流水功能实现

AI 查流水对应 transactions 意图。用户可以在自然语言中给出交易类型、时间范围和条数限制。例如，“查最近 3 条支出记录”会解析为 txnType= 支出、limit=3、timeRange=latest；“查今天的流水”会解析为 timeRange=day；“查前几天的支出记录”会经过 Time.normalizeTimeByText 修正为 recent_days。

handleTransactions 根据解析结果构造 TransactionQueryDTO，并调用

Time.fillTimeRange 填充 startTime 和 endTime。随后系统调用 TransactionServiceImpl.getTransactions 查询流水，底层 SQL 由 TransactionMapper.xml 中的 select-Transactions 完成。查询结果再由 convertTransactionResultToChat 格式化为自然语言列表，并对对方账号进行脱敏。

对于缺少月份的日期查询，系统设计了多轮补全机制。当 needMonth 为 true、day 存在但 month 为空时，当前查询会被保存到 pendingTxnQueryMap，系统提示用户补充月份。用户回复“1月”后，下一轮请求会优先进入 handlePendingTransactionQuery，而不是重新当作普通问题解析。该方法读取待补全状态、解析月份、构造 TransactionQueryDTO，并继续调用流水查询服务。

流水查询比余额查询更能体现自然语言到结构化条件的转换。用户表达中可能同时包含交易方向、相对时间、绝对日期和条数限制，后端需要把这些信息映射为 TransactionQueryDTO 字段。由于交易流水又是账单统计和历史金额复用的基础数据，查询条件是否准确会直接影响后续结果。本文实现中采用模型解析与规则纠偏结合的方式：模型理解用户表达，Time 工具修正时间范围，Mapper 层执行确定性 SQL 查询。

图6.2 用时序图展示缺少月份时的两轮交互。第一轮中，用户提出“查 17 号收入记录”，系统识别为 transactions 意图，并发现 month 为空，于是保存 PendingTransactionQuery 并追问月份；第二轮中，用户回复“1月”，AssistantService 优先命中 pendingTxnQueryMap，补齐日期范围后调用 TransactionService 和 TransactionMapper 完成流水查询。

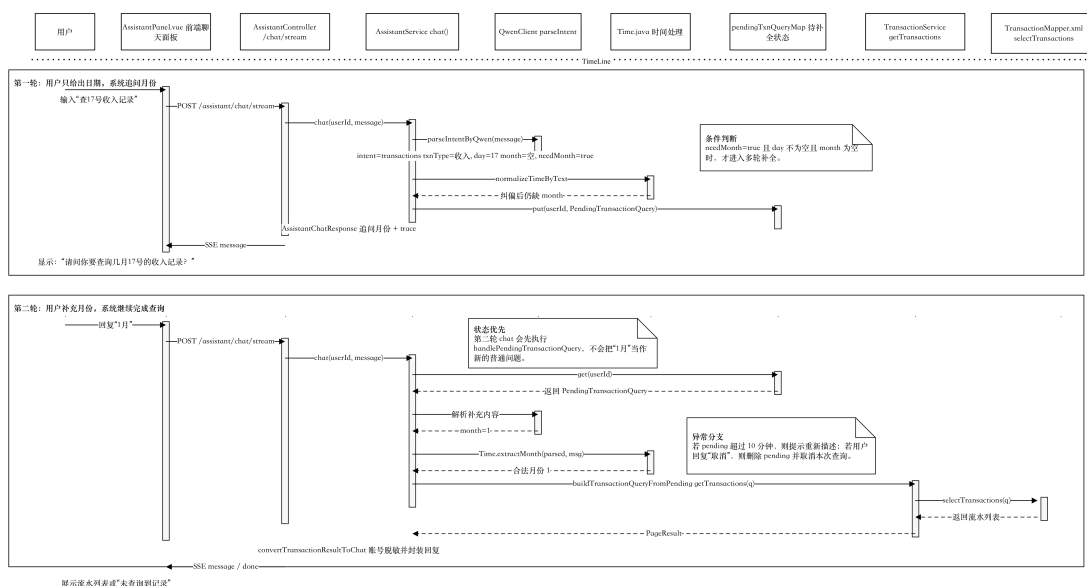


图 6.2 AI 查流水多轮补全时序图

表 6.2 AI 查流水测试输入与处理方式

用户输入	关键解析字段	系统行为
查最近 3 条支出记录	txnType= 支 出, limit=3, timeRange=latest	查询最近 3 条支出流水。
查今天的流水	timeRange=day	查询当天全部流水。
查前几天的支出记录	txnType= 支 出, timeRange=recent_days, rangeValue=3	查询近 3 天支出流水。
查 17 号收入记录	txnType= 收 入, day=17, need-Month=true	追问月份并进入待补全状态。

6.7 AI 账单统计与分析功能实现

AI 账单统计与分析对应 `statistics` 意图。系统支持 `sum`、`count`、`max` 和 `trend` 四类指标，分别用于统计总金额、流水笔数、最大一笔交易和趋势数据。用户输入“统计这个月总支出”时，系统构造 `txnType=支出`、`metric=sum`、`timeRange=month` 的 `StatisticsQueryDTO`；用户输入“看最近 7 天支出趋势”时，系统构造 `txnType=支出`、`metric=trend`、`timeRange=recent_days`、`groupBy=day` 的查询对象。

表 6.3 统计指标与实现方法对应关系

统计指标	含义	底层方法	前端展示
sum	总金额统计	sumAmount	数字结果或收支概览图。
count	笔数统计	countByStatistics	数字结果。
max	最大一笔统计	maxTransaction	数字结果。
trend	趋势统计	statisticsTrend	柱状图或折线图。

`StatisticsServiceImpl` 根据 `metric` 调用不同 `Mapper` 方法：`sumAmount` 计算总额，`countByStatistics` 统计笔数，`maxTransaction` 查找最大一笔交易，`statisticsTrend` 完成趋势聚合。当用户查询整体收支概览时，系统还会通过 `sumIncomeAndExpense` 和 `statisticsByCounterparty` 生成收支对比以及按对方账号聚合的图表数据。因此，账单统计与分析本质上是在交易流水查询基础上的进一步聚合和解释：流水查询返回明细，统计分析则在同一数据基础上形成汇总、趋势和概览。

统计分析文本采用“事实构造 + 模型总结 + 规则化后备总结”的生成方式。`AssistantService` 先把统计标题、统计类型、统计指标、时间范围、统计值、趋势明细、主要对方账号和异常信号等内容整理为 `facts`，再请求通义千问生成自然语

言总结。若模型输出为空、过于模板化，或包含不支持的标签、消费类别等内容，系统会改用规则化后备总结。与多轮查流水不同，统计分析关注的不是消息时序，而是自然语言条件如何转换为统计 DTO，并经过确定性 SQL 聚合、事实整理和模型表达形成完整分析结果。

图6.3 展示 AI 账单统计与分析的整体流程。系统先解析 `statistics` 意图并构造 `StatisticsQueryDTO`，再由 `StatisticsServiceImpl` 和 `TransactionMapper.xml` 完成总额、笔数、最大值或趋势等确定性统计。统计结果封装为 `StatisticsResultVO` 后，`AssistantService` 继续整理 `facts` 并调用大模型生成分析总结；当模型输出不符合约束时，系统使用规则化后备总结，最后将 `reply`、`chartData`、`explain` 和 `exportUrl` 等字段返回前端。

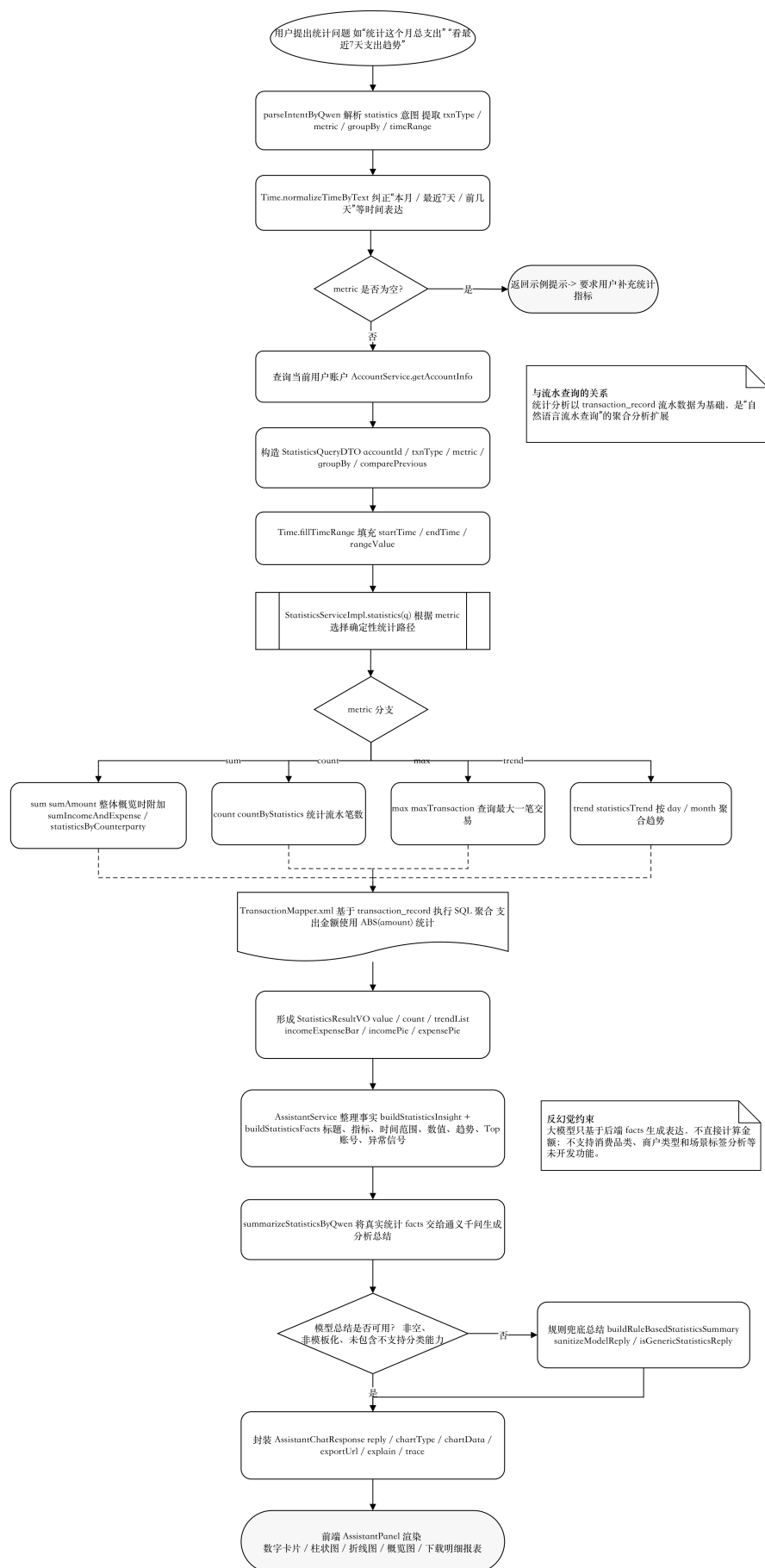


图 6.3 AI 账单统计与分析渲染图

前端根据 `AssistantChatResponse` 中的 `chartType` 和 `chartData` 渲染图表。`chartType` 为 `number` 时显示数字卡片，为 `bar` 或 `line` 时显示趋势图，为 `overview` 时显示收支对比柱状图，以及收入、支出按对方账号聚合的饼图。系统还会生成 `exportUrl`，用户可下载当前统计口径下的交易流水 Excel 文件。



图 6.4 AI 账单统计与图表展示界面图

6.8 AI 周报/月报功能实现

AI 周报/月报由 `/assistant/brief` 接口提供，主要用于用户打开 AI 面板时自动展示财务概览。该功能不依赖用户主动提问，而是根据 `period` 参数生成 `week` 或 `month` 简报。

`getBrief` 方法先查询当前用户账户，再构造收入总额、支出总额和支出趋势三个统计查询。系统计算总收入、总支出和净流入，并生成与上周或上月的对比数据。异常提醒方面，系统以近 4 周平均支出和近 30 天支出金额 P90 阈值作为参考，判断当前周期支出是否明显偏高，或是否存在超大单笔支出。

前端 `AssistantPanel` 在 `mounted` 阶段调用 `fetchAssistantBriefApi` 加载简报，并在

面板顶部展示 headline、highlights、suggestions、anomalies、comparison、quickActions 和 chartData。用户可以切换周报和月报，也可以点击快捷追问按钮继续向 AI 发起查询。

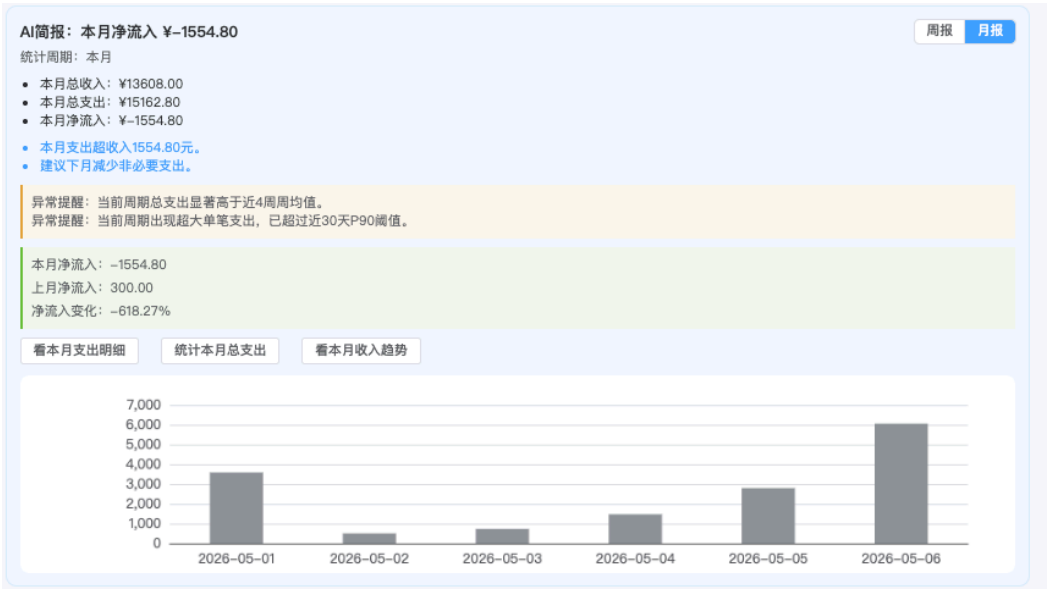


图 6.5 AI 月报卡片界面图

表 6.4 AI 周报/月报返回字段说明

简报字段	含义
headline	简报标题，例如“AI 简报：本月净流入”。
highlights	本周或本月总收入、总支出、净流入。
comparison	当前周期与上一周期的收入、支出、净流入对比。
anomalies	异常支出提醒。
suggestions	AI 生成的简短建议。
quickActions	可直接点击发送的追问指令。
chartData	支出趋势图数据。

6.9 AI 智能转账功能实现

AI 智能转账对应 transfer 意图，是系统中安全要求最高的 AI 功能。用户可以直接提供银行卡号，也可以使用收款人昵称。例如，“向 6222000012345678 转 100 元”会提取 toAccountNo 和 amount；“给弟弟转 300 元，备注午餐费”会提取 payeeAlias= 弟弟、amount=300、description= 午餐费。

handleTransfer 首先解析收款账号、收款人昵称、金额、备注和 repeatMode。如果用户没有提供银行卡号但提供了昵称，系统会通过 PayeeContactMap-

`per.selectByUserIdAndAlias` 查询当前用户的收款人列表。昵称未找到时，系统提示用户先新增收款人或直接提供银行卡号；若匹配到多个收款人，系统提示用户使用更具体的昵称。

金额解析完成后，系统不会立即扣款，而是调用 `accountService.validateTransfer` 或 `payeeService.validateTransfer` 进行校验。校验内容包括参数完整性、金额是否大于 0、付款账户是否存在、收款账户是否存在、账户是否冻结、是否给自己转账以及余额是否充足。校验通过后，系统把转账信息写入 `PendingTransfer`，并向用户返回脱敏后的确认信息。只有当用户回复“确认”或点击“确认”按钮后，`handlePendingTransfer` 才会调用真实转账服务执行扣款、收款和流水写入；用户回复“取消”或点击“取消”按钮时，系统删除待确认状态。

AI 智能转账涉及两轮用户交互和资金操作确认，如图 6.6 所示。第一轮中，用户通过自然语言发起转账，`AssistantService` 调用 `QwenClient` 解析 `transfer` 意图，并根据银行卡号或收款人昵称完成收款人匹配；随后系统调用转账校验服务检查账户状态、余额和自转账等风险条件。校验通过后，系统只生成确认信息并保存 `PendingTransfer`，不执行资金变动。第二轮中，用户回复“确认”或点击“确认”按钮时，系统才调用真实转账服务更新账户余额并写入交易流水；用户回复“取消”、点击“取消”按钮或待确认状态超时，系统清理 `pendingMap`，资金操作不会发生。

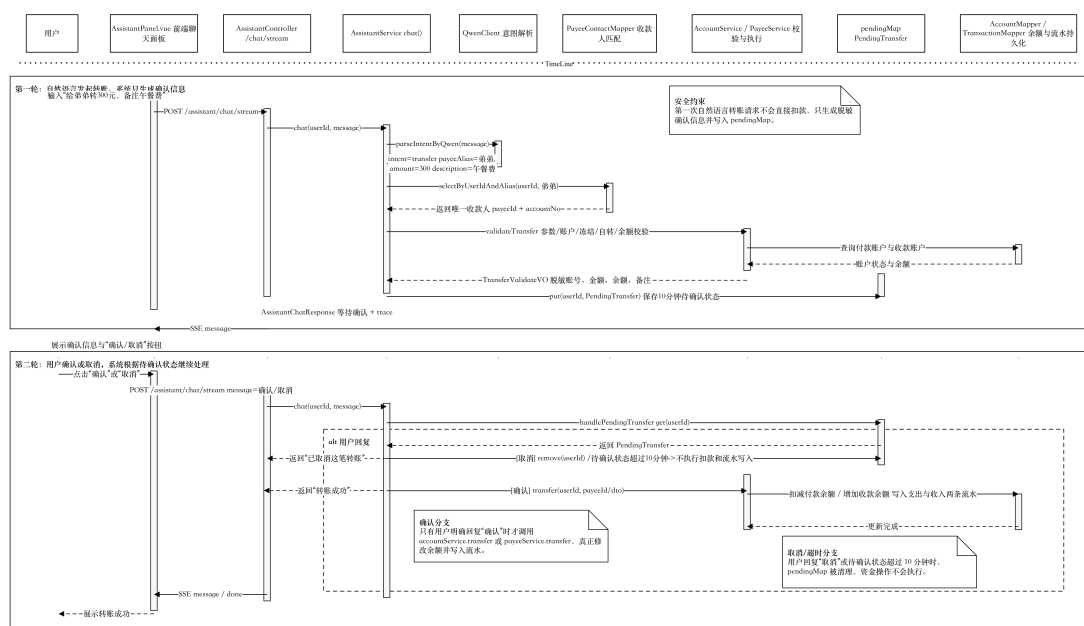


图 6.6 AI 智能转账二次确认时序图



图 6.7 AI 智能转账二次确认界面图

系统还实现了历史金额复用能力。当用户输入给弟弟转晚餐费并说明“和前天一样”时，模型会把 `repeatMode` 解析为 `time_same`；后端再通过 `Time` 工具确定参考时间范围，并调用 `selectLatestExpenseByCounterpartyAndDescription` 查询该时间范围内同收款人且备注匹配的历史支出流水。若找到记录，系统复用历史金额和备注生成确认信息；若未找到，则提示用户补充转账金额。该能力只用于用户未明确给出金额时的补全，不会绕过转账校验和二次确认。

图6.8 展示历史金额复用转账的处理流程。系统先判断当前 `transfer` 请求是否属于 `amount` 为空且包含时间参考的复用场景，再解析收款人、备注关键词和参考时间范围。查询历史流水时，系统优先查找同收款人、同备注关键词的支出记录；若未找到，再回退查询同收款人的最近支出流水。只有查到真实历史流水时，系统才复用其金额和备注，否则要求用户补充金额。

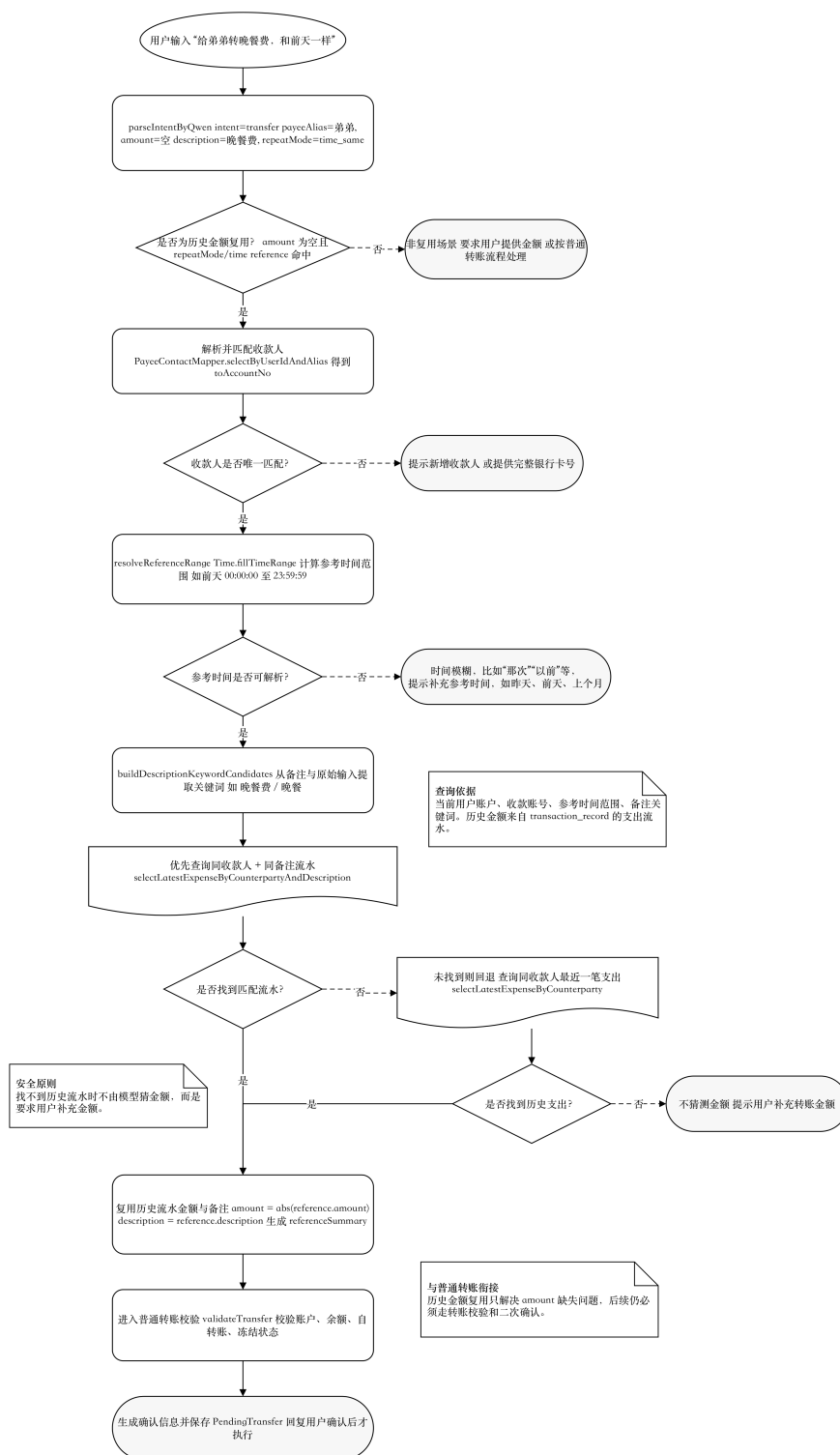


图 6.8 历史金额复用转账流程图

表 6.5 AI 智能转账安全控制设计

安全控制点	实现方式
收款人确认	按银行卡号或收款人昵称解析，昵称不存在或重复时提示用户补充。
账户校验	校验付款账户、收款账户是否存在及状态是否正常。
余额校验	转账前检查付款账户余额是否充足。
防止自转账	付款账号与收款账号相同时抛出业务异常。
二次确认	AI 首次回复只生成确认信息，用户回复确认后才执行。
账号脱敏	确认信息中付款账号和收款账号均以脱敏形式展示。

第七章 系统测试与交互优化分析

本章对应前文提出的“可验证评价方式”和“测试与反馈闭环”。本章重点不是再次描述功能，而是说明系统实现效果能否由测试过程和数据记录支撑。7.1 介绍测试环境、数据来源、能力边界和指标定义；7.2 用后端自动化测试检查核心服务分支；7.3 验证大模型意图解析和服务分流效果；7.4 通过功能测试、FAQ 检索统计和边界用例检查核心功能；7.5 基于匿名反馈测试日志分析用户体验；7.6 和 7.7 再把测试结果回扣到交互优化、效率提升、能力边界和后续改进方向。

7.1 测试环境与测试方法

系统测试基于本地开发环境进行，后端服务运行在 8081 端口，前端通过 Vite 开发服务器将 /api 请求代理到后端。数据库采用 MySQL，测试数据来自 sql 目录中的 user、account、payee_contact 和 transaction_record 脚本，数据库内的信息均为演示数据，不涉及真实客户。测试方法以功能测试和交互流程测试为主，重点观察 AI 助手能否识别用户输入、调用对应业务服务，并返回符合预期的文本、图表和确认流程。

本项目测试数据由系统初始化脚本中的虚拟账户、虚拟交易流水、FAQ 初始化数据，以及校外企业导师提供的测试建议共同构成。测试过程不使用真实银行客户信息，账号、姓名、手机号、余额、交易流水和问答内容均为模拟或脱敏数据。交互优化分析则主要从功能可用性、任务完成情况和体验反馈三个角度展开。

测试范围围绕本文定义的 AI 核心功能展开，登录、注册、普通页面跳转等基础管理能力不作为主要评价对象。测试开始前先确认后端服务、数据库和前端代理能够正常启动，再在 AI 面板中输入典型自然语言指令，对照后端控制台、浏览器界面和接口返回结果。语音输入重点检查转写结果能否回填输入框并继续发送；查询类功能重点检查金额、记录条数、交易方向和时间范围是否与数据库样本一致；统计类功能重点检查统计口径、聚合结果、图表数据和导出链接是否完整；转账类功能重点检查系统是否先生成确认信息，以及是否在用户明确确认前避免修改账户余额。

为保证测试结论与源代码一致，本文不设计超出项目能力范围的测试项。例如，当前系统没有实现商户品类识别和消费场景标签，因此测试只检查系统能否拒绝或说明该类请求，而不把它写成已完成能力。考虑到大模型输出存在一定随机性，测试判断主要依据结构化字段和业务服务调用结果，而不是要求每次自然

语言回复完全一致。这样可以避免把措辞差异误判为功能失败，也能更接近系统真实可用性。

7.1.1 测试指标定义与计算方法

为避免测试结果只停留在人工描述和表格罗列，本文在测试阶段分别为后端自动化测试、意图识别与服务分流测试、FAQ 知识库检索测试和用户体验测试设置量化指标。设某类测试样本集合为

$$D = \{d_1, d_2, \dots, d_n\},$$

其中 n 表示样本总数。对于第 i 个测试样本，若系统输出结果与人工期望结果一致，则记为

$$I_i = \begin{cases} 1, & \text{系统输出符合预期;} \\ 0, & \text{系统输出不符合预期.} \end{cases}$$

因此，某类测试任务的通过率定义为

$$P = \frac{\sum_{i=1}^n I_i}{n} \times 100\%.$$

该指标用于功能测试、能力边界测试以及意图识别与服务分流测试，用来衡量系统在给定样本集合上的正确处理比例。

对于后端自动化测试，设测试用例总数为 N ，通过用例数为 N_p ，失败用例数为 N_f ，跳过用例数为 N_s ，则通过率、失败率和跳过率分别定义为

$$P_{pass} = \frac{N_p}{N} \times 100\%,$$

$$P_{fail} = \frac{N_f}{N} \times 100\%,$$

$$P_{skip} = \frac{N_s}{N} \times 100\%.$$

设第 k 个测试类的执行耗时为 t_k ，测试类数量为 m ，则自动化测试总耗时和平均耗时分别定义为

$$T_{total} = \sum_{k=1}^m t_k,$$

$$\bar{T} = \frac{1}{m} \sum_{k=1}^m t_k.$$

对于意图识别与服务分流测试，设第 i 条测试语句的人工期望分支为 y_i ，系统实际分流结果为 $\hat{y}_i$ 。若二者一致，则认为该样本通过。整体分流准确率定义为

$$Acc = \frac{1}{n} \sum_{i=1}^n \mathbf{1}(\hat{y}_i = y_i) \times 100\%,$$

其中 $\mathbf{1}(\cdot)$ 为指示函数, 当括号内条件成立时取 1, 否则取 0。对于第 c 类场景, 设该类样本数为 n_c , 通过数为 p_c , 则该类场景通过率为

$$Acc_c = \frac{p_c}{n_c} \times 100\%.$$

对于 FAQ 知识库检索测试, 系统检索结果分为直接命中、相似推荐和未命中三类。设 FAQ 测试问题总数为 N_{faq} , 直接命中数量为 N_{hit} , 相似推荐数量为 N_{rec} , 未命中数量为 N_{no} , 则直接命中率、相似推荐率和未命中率分别为

$$\begin{aligned} R_{hit} &= \frac{N_{hit}}{N_{faq}} \times 100\%, \\ R_{rec} &= \frac{N_{rec}}{N_{faq}} \times 100\%, \\ R_{no} &= \frac{N_{no}}{N_{faq}} \times 100\%. \end{aligned}$$

对于银行业务相关问题, 本文将直接命中和相似推荐均视为有效处理, 相关问题覆盖率定义为

$$R_{cover} = \frac{N_{hit}^{bank} + N_{rec}^{bank}}{N_{bank}} \times 100\%,$$

其中 N_{bank} 表示银行业务相关 FAQ 测试样本数量。对于无关问题, 若系统进入未命中分支而不是直接编造银行业务答案, 则视为有效拦截, 无关问题拦截率定义为

$$R_{reject} = \frac{N_{no}^{irrelevant}}{N_{irrelevant}} \times 100\%.$$

对于用户体验测试, 设第 j 个任务包含 n_j 条测试记录。第 i 条记录的完成状态为 c_{ij} , 完成记为 1, 未完成记为 0, 则任务完成率定义为

$$C_j = \frac{\sum_{i=1}^{n_j} c_{ij}}{n_j} \times 100\%.$$

设第 i 条记录的响应耗时为 d_{ij} , 单位为毫秒, 则第 j 个任务的平均响应耗时为

$$\bar{T}_j = \frac{1}{n_j} \sum_{i=1}^{n_j} \frac{d_{ij}}{1000}.$$

设第 i 条记录的操作步数为 s_{ij} , 则第 j 个任务的平均操作步数为

$$\bar{S}_j = \frac{1}{n_j} \sum_{i=1}^{n_j} s_{ij}.$$

对于理解度、满意度、安全感和语言表达四项主观评分, 评分范围均为 1–5 分。设第 i 条记录在第 q 个评分维度上的分值为 r_{ijq} , 则第 j 个任务在该维度上的平均评分为

$$\bar{R}_{jq} = \frac{1}{n_j} \sum_{i=1}^{n_j} r_{ijq}.$$

分值越高，说明测试者对该任务的理解程度、满意程度、安全感或语言表达认可程度越高。

7.2 后端自动化测试与指标记录

考虑到单纯人工体验测试样本量较小、数据稳定性有限，本文补充了 JUnit 5 和 Mockito 自动化测试。自动化测试不直接连接真实大模型服务和真实银行数据库，而是通过 Mock 对象构造固定返回值，重点检查业务分支、参数解析、知识库检索和日志记录逻辑。这样在代码修改后可以重复执行同一批用例，避免仅依赖一次性人工操作截图或主观反馈。

自动化测试覆盖范围如表 7.1 所示。这里的测试重点并不是证明系统已达到生产级稳定性，而是验证论文中描述的核心后端逻辑确实存在，并且能在可控输入下得到预期输出。

表 7.1 后端自动化测试覆盖范围

测试对象	测试类简称	覆盖内容	用例数	结果
应用入口	应用启动测试	应用主类可构造，避免基础类加载失败。	1	通过
时间解析	时间工具测试	本月范围、最近天数范围、昨天等自然语言时间归一化。	3	通过
统计服务	统计服务测试	支出合计、趋势列表、空结果处理。	3	通过
转账校验	账户服务测试	正常转账校验、余额不足、向本人转账拦截。	3	通过
FAQ 检索	知识库测试	直接命中、相似推荐、未命中、查询日志写入。	4	通过
AI 编排流程	助手服务测试	情绪安抚、余额查询、FAQ 问答、转账取消、多轮补全。	5	通过
指标日志	日志服务测试	trace 信息提取、日志表不可用时不影响主流程。	2	通过
合计	—	覆盖 AI 助手主要后端分支。	21	通过

测试命令为 mvn test，测试报告由 Maven Surefire 插件生成。2026 年 4 月 30 日在本地环境执行的结果见表 7.2。表中“用例耗时”来自 Surefire 报告中各测试类 time 字段的单项值或合计值；Maven 构建总耗时还包括依赖解析、编译和测试框架启动时间。

表 7.2 后端自动化测试执行结果

测试类简称	用例数	失败数	跳过数	耗时/s
应用启动测试	1	0	0	0.084
时间工具测试	3	0	0	0.304
统计服务测试	3	0	0	1.693
账户服务测试	3	0	0	0.101
日志服务测试	2	0	0	0.523
知识库测试	4	0	0	0.086
助手服务测试	5	0	0	0.733
合计	21	0	0	3.524

按照前文自动化测试指标定义，本次后端自动化测试共执行 21 个用例，其中 21 个通过，失败和跳过均为 0。因此自动化测试通过率为

$$P_{pass} = \frac{21}{21} \times 100\% = 100.0\%,$$

失败率和跳过率均为

$$P_{fail} = P_{skip} = 0.$$

各测试类耗时相加得到总耗时

$$T_{total} = 0.084 + 0.304 + 1.693 + 0.101 + 0.523 + 0.086 + 0.733 = 3.524s.$$

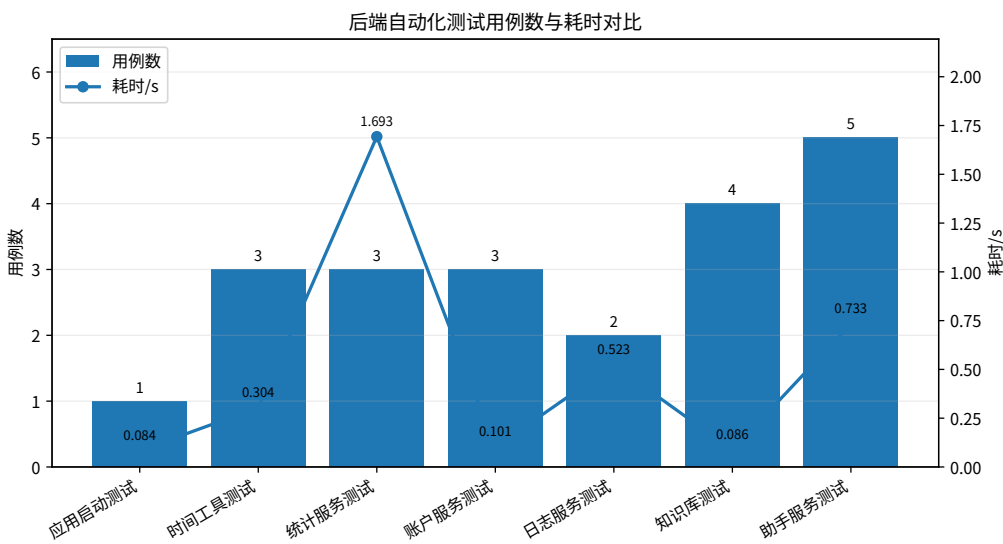


图 7.1 后端自动化测试用例数与耗时对比

从图 7.1 可以看到，助手服务测试和知识库测试覆盖用例数相对较多，统计服务测试耗时最高。各测试类均未出现失败或跳过，说明后端核心分支在可控输入下能够稳定通过。

除测试报告外，后端还增加 `assistant_test_log` 指标日志表，用于记录 AI 助手请求运行结果。主要字段中，`duration_ms` 用于统计接口处理耗时，`success`、`result_code` 和 `biz_code` 用于统计成功率和异常类型，`intent` 记录系统实际识别出的业务意图，`trace_tool` 和 `trace_status` 用于分析系统实际调用了余额查询、流水查询、统计服务、FAQ 检索还是转账确认流程。为支撑匿名反馈测试，表中扩展了 `test_session_id`、`participant_code`、`task_code`、`task_name`、`completed`、`operation_steps`、`understanding_score`、`satisfaction_score`、`safety_score`、`language_score` 和 `user_feedback` 等字段。这些字段只在匿名反馈环节记录，其中 `task_code` 和 `task_name` 来自前端 U1-U8 任务选择；`operation_steps` 只记录前端可观察的关键动作，如选择任务、填入示例、启动语音输入、发送消息以及转账确认或取消，不记录无法准确观测的键盘输入字符数。四类评分采用 1-5 整数值，并随每条任务聊天记录写入，而不是只在测试结束时记录一次总体评分。

该日志表的作用，是把用户体验测试中的“完成时间、是否成功、调用路径是否正确”等观察项转为后端可记录字段。人工测试仍用于观察用户是否理解回复、是否愿意继续使用等体验反馈；自动化测试和日志表则补充客观证据，说明测试数据可以从可重复执行的测试报告 and 后端运行记录中获得，而不只依赖主观描述。

7.3 意图识别准确性测试

为了验证“大语言模型意图解析 + FAQ 检索 + 规则化后备处理 + 情绪关键词检测”方案是否满足任务书中“选择适用的技术和算法”的要求，本文对 `AssistantService` 的意图识别和分流链路进行了测试。需要说明的是，`parseIntentByQwen` 只要求模型输出 `balance`、`transactions`、`statistics`、`transfer` 和 `other` 五类基础意图；FAQ 咨询、异常情绪安抚和待确认状态处理不是模型直接输出的新意图，而是在服务编排层继续分流得到的处理结果。

从源码流程看，请求进入 `AssistantService` 后，系统先处理待确认转账和待补全流水查询，再通过 `handleEmotionSupport` 对明显投诉、愤怒或焦虑表达做关键词检测；上述规则均未命中时，系统才调用 `parseIntentByQwen` 解析基础意图。对于 FAQ 咨询、未实现能力和非银行业务问题，模型层统一输出 `other`，随后由 `handleFaqOrFallback` 进入知识库检索或后备回复。因此，本节测试既记录模型层 `intent` 是否正确，也记录最终 `trace` 中的工具调用和状态是否符合预期。

测试样本覆盖余额查询、流水查询、账单统计、转账发起、FAQ 咨询、异常情绪安抚、不支持能力拦截和多轮状态处理八类场景，每类 10 条中文表达，共 80 条。样本既包含“查一下我的余额”“统计这个月总支出”等直接表达，也包

含“我现在账户里还有多少钱”“最近几天的交易明细给我看看”“给弟弟转晚餐费，和昨天一样”等口语化表达。FAQ 咨询样本包括“卡丢了怎么挂失”“跨行转账要多久”等常见问题；情绪样本包括“我很生气，我要投诉”“我有点担心这笔钱”等表达；不支持能力样本包括“按消费场景标签统计一下”“按商户类型分析消费”等超出当前系统能力边界的问题。

表 7.3 AI 助手意图与分流测试结果

场景类型	期望处理分支	测试语句数	通过数	通过率	典型测试语句
查余额	balance / account_info	10	10	100.0%	查一下我的余额
查流水	transactions / transaction_query	10	9	90.0%	查最近 5 条支出记录
账单统计	statistics / statistics_service	10	9	90.0%	最近 7 天我花了多少钱
转账发起	transfer / transfer_validation	10	9	90.0%	给弟弟转 300 元午餐费
FAQ 咨询	other 后 进入 faq_knowledge_base	10	10	100.0%	卡丢了怎么挂失
情绪安抚	emotion_support rule_emotion	10	10	100.0%	我很生气，我要投诉
不支持能力	other / qwen_fallback	10	9	90.0%	按消费场景标签统计一下
多轮与确认状态	pending 状态续接或取消	10	10	100.0%	1 月；确认；取消
合计	—	80	76	95.0%	典型样本集合

根据意图识别与服务分流准确率公式，本次共构造 80 条典型样本，其中 76 条的实际分流结果与人工期望一致，因此整体分流准确率为

$$Acc = \frac{76}{80} \times 100\% = 95.0\%.$$

其中查流水、账单统计、转账发起和不支持能力拦截场景的单类通过率为 90.0%，余额查询、FAQ 咨询、情绪安抚和多轮确认场景的单类通过率为 100.0%。

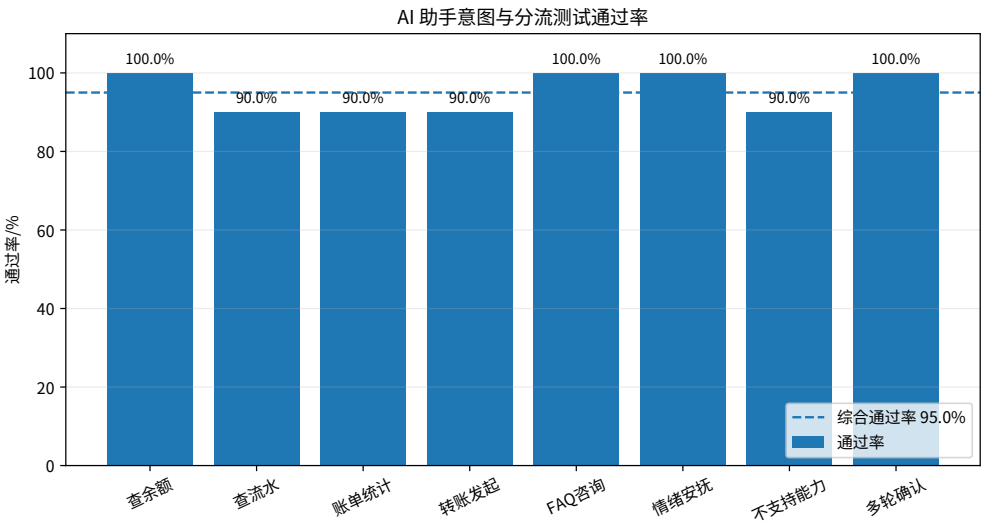


图 7.2 AI 助手意图与分流测试通过率

图 7.2 展示不同场景下 AI 助手意图识别与服务分流的通过率。余额查询、FAQ 咨询、情绪安抚和多轮确认场景均达到 100.0%，查流水、账单统计、转账发起和不支持能力拦截场景为 90.0%。这说明系统在高频且边界明确的场景中较稳定，而模糊表达和超出能力边界的请求仍需继续优化。结合表 7.3，本次 80 条典型样本中系统通过 76 条，综合通过率为 95.0%。未通过样例及原因见表 7.4。

表 7.4 AI 助手意图与分流误判样例

测试样例	人工期望分支	实际模型或分流结果	问题原因	优化方向
最近几天花钱情况给我看看	statistics	transactions	“花钱情况”既可理解为支出明细，也可理解为支出汇总，语义边界不够明确。	提示词强调“花钱情况、消费情况”优先进入统计。
查一下这个月比较大的几笔收入	transactions	statistics	“比较大的”触发了最大值统计倾向，但用户实际希望查看多条收入流水。	增加“大几笔、几条记录”优先流水查询规则。
给弟弟转晚餐费，和那天一样	transfer	other	“那天一样”缺少明确日期，历史金额复用所需的参考时间不完整。	增加追问日期逻辑和 repeat-Mode 示例。
按吃饭购物场景统计一下本月支出	other qwen_fallback	/ statistics	当前系统没有消费场景标签字段，但模型容易把“统计”关键词识别为已支持统计。	扩展不支持能力关键词和后置拦截规则。

测试结果说明，在本文设定的银行客服功能边界内，基础意图解析配合 FAQ 知识库、关键词规则和状态机后备处理，可以满足原型系统的基本需求。误判主要集中在表达模糊、同时包含多个业务关键词或请求超出系统能力边界的场景。例如，“花钱情况”可能指流水明细，也可能指支出统计；“比较大的几笔收入”可能被理解为多条流水，也可能被理解为最大值统计；“吃饭购物场景”虽然是常见用户表达，但当前数据库没有对应标签字段，仍需依靠 containsUnsupportedScope 和后置规则拦截。该结果也说明，系统不是把所有请求交给大模型自由处理，而是通过后端编排层把 FAQ、情绪安抚和风险确认纳入更可控的分流链路。

需要说明的是，该结果不能说明系统对任意开放式自然语言问题都能保持同等准确率。本次样本主要围绕已实现功能和答辩演示高频问法构造，样本构成见附录 A.4，目的是验证源码中的意图解析提示词、JSON 输出约束、FAQ 分流、情绪关键词检测、状态续接和不支持能力拦截是否有效。由于样本来自已实现功能边界内的典型问法，该结果不能外推为开放域自然语言理解准确率。若面向更大规模真实用户语料，还需要扩大样本数量，并引入准确率、召回率、混淆矩阵和人工复核等更完整的评估方法。

7.4 功能测试

表 7.5 AI 核心功能测试用例表

编号	测试输入或操作	预期结果	结论
T1	输入“查一下我的余额”	识别为 balance ，返回账户信息和余额。	通过
T2	输入“查最近 5 条支出记录”	识别为 transactions ，返回支出流水列表。	通过
T3	输入“查 17 号收入记录” 后回复“1 月”	系统先追问月份，再查询指定日期收入流水。	通过
T4	输入“统计这个月总支出”	识别为 statistics ，返回支出总额、统计口径和导出链接。	通过
T5	输入“看最近 7 天支出趋势”	返回趋势图数据，前端可渲染柱状图或折线图。	通过
T6	打开 AI 面板	自动加载周报或月报卡片。	通过
T7	输入“给弟弟转 300 元，备注午餐费”	按昵称匹配收款人并返回转账确认信息。	通过
T8	在待确认转账状态下输入“取消”	取消转账并清除待确认状态。	通过
T9	输入“给弟弟转晚餐费，和昨天一样”	识别为 transfer ，按昵称匹配收款人，并根据历史流水复用金额和备注，生成待确认转账信息。	通过
T10	提出消费分类或商户类型分析请求	系统说明当前不支持该类能力。	通过
T11	点击语音输入按钮并说出“查一下我的余额”	浏览器完成语音转写，输入框出现识别文本，发送后进入余额查询流程。	通过
T12	输入“我很生气，我要投诉”	系统识别异常情绪或投诉表达，先进行安抚，并提示拨打虚拟客服电话 010-1234-5678 联系人工客服。	通过
T13	输入“卡丢了怎么挂失”	系统进入 FAQ 知识库检索，命中银行卡挂失问题并返回标准答案。	通过
T14	输入“转账有问题”	系统未直接确定唯一答案时，返回转账相关的相似问题推荐。	通过

为进一步检查系统不会把未实现能力或高风险操作包装成正常结果，本文补充能力边界与安全回退测试，见表 7.6。该表重点观察系统在无法回答、不能执行或需要用户确认时，能否进入明确的拒答、推荐、追问、取消或安抚分支。

表 7.6 AI 能力边界与安全回退测试

测试输入或场景	预期边界行为	实现机制	结论
按消费场景标签统计本月支出	不生成虚假的分类统计结果，提示当前不支持该能力。	containsUnsupportedScope 拦截消费分类、品类、商户类型和场景标签类请求。	通过
今天西安天气怎么样	不编造银行业务答案，进入 FAQ 未命中或后备回复。	FAQ 检索 no_answer 后进入 fallbackChat，并限制回答范围。	通过
卡丢了怎么挂失	不由模型自由编写银行规则，优先返回知识库标准答案。	handleFaqOrFallback 调用 faq_knowledge 表，并写入 faq_query_log。	通过
给弟弟转 300 元午餐费	不直接扣款，只生成待确认转账信息。	handleTransfer 完成收款人匹配和校验后写入 pendingMap，等待确认。	通过
待确认转账后输入“取消”	清除待确认状态，不继续执行资金操作。	handlePendingTransfer 识别取消意图并移除 pendingMap。	通过
我很生气，我要投诉	不继续普通业务闲聊，先安抚并引导人工客服。	handleEmotionSupport 在调用模型前进行关键词检测。	通过

从测试结果看，系统能够覆盖论文设计的语音输入、FAQ 知识库问答、异常情绪安抚和五项核心 AI 业务功能。语音输入任务中，系统可以把语音转换为可编辑文本；FAQ 咨询中，系统可以从知识库返回标准答案或相似问题推荐；异常情绪或投诉表达中，系统会先安抚用户并提示人工客服渠道；常规查询类任务中，系统可通过意图解析进入对应业务流程；缺失信息的查询可以通过多轮对话补全；转账类任务则会在执行前给出确认信息并等待用户明确操作。

语音输入测试主要验证前端输入入口。用户点击语音输入按钮后，浏览器请求麦克风权限并启动 SpeechRecognition；识别到中文语音后，前端将转写文本回填到 chatInput，用户可以继续修改或直接发送。测试表明，语音识别已接入智能客服入口，但后续业务处理仍遵循文本请求链路，不会绕过后端意图解析和安全校验。

异常情绪安抚测试主要验证客服交互边界。用户输入明显包含投诉、愤怒或焦虑的表达时，后端先触发 handleEmotionSupport，返回安抚话术和人工客服电话，而不是继续进入普通业务意图解析。测试结果表明，该模块能识别明显负面情绪，并把复杂服务纠纷引导到人工客服渠道。由于它不参与账户和转账业务执行，所以不会改变系统资金安全边界。

FAQ 知识库测试主要验证常见咨询能否从数据库返回确定答案。用户输入“卡丢了怎么挂失”“跨行转账要多久”“验证码收不到怎么办”等问题时，系统先进入 FAQ 检索流程，命中后返回 faq_knowledge 表中的标准答案；对于“转账有问题”等模糊表达，系统返回相似问题推荐；对于明显无关问题，系统记录 no_answer 后进入后备回复。通过 faq_query_log 表可以统计命中、推荐和未命中数量，使实验部分不只停留在“通过/不通过”的描述。

为进一步检查 FAQ 知识库模块效果，本文从知识库测试集中选取银行卡业

务、转账业务、密码与安全、贷款咨询和无关问题五类样本，每类 10 条，共 50 条问题进行检索测试。测试结果按照 `faq_query_log` 中的 `result_type` 字段统计，如表 7.7 所示。

表 7.7 FAQ 知识库检索统计结果

测试类别	测试问题数	直接命中	相似推荐	未命中	说明
银行卡业务	10	8	2	0	挂失、补卡、吞卡、交易提醒等。
转账业务	10	7	3	0	到账时间、失败原因、限额、填错账号等。
密码与安全	10	8	1	1	密码找回、验证码、诈骗防范、异常交易等。
贷款咨询	10	7	2	1	贷款材料、进度、提前还款、利率和逾期等。
无关问题	10	0	1	9	天气、机票、外卖、旅游、论文格式等。
合计	50	30	9	11	相关问题优先返回答案或推荐，无关问题避免直接编造答案。

根据 FAQ 检索指标定义，本次 FAQ 测试共 50 条样本，其中直接命中 30 条、相似推荐 9 条、未命中 11 条，因此直接命中率、相似推荐率和未命中率分别为

$$R_{hit} = \frac{30}{50} \times 100\% = 60.0\%,$$

$$R_{rec} = \frac{9}{50} \times 100\% = 18.0\%,$$

$$R_{no} = \frac{11}{50} \times 100\% = 22.0\%.$$

在 40 条银行业务相关问题中，直接命中 30 条，相似推荐 8 条，因此相关问题覆盖率为

$$R_{cover} = \frac{30 + 8}{40} \times 100\% = 95.0\%.$$

在 10 条无关问题中，9 条进入未命中分支，因此无关问题拦截率为

$$R_{reject} = \frac{9}{10} \times 100\% = 90.0\%.$$

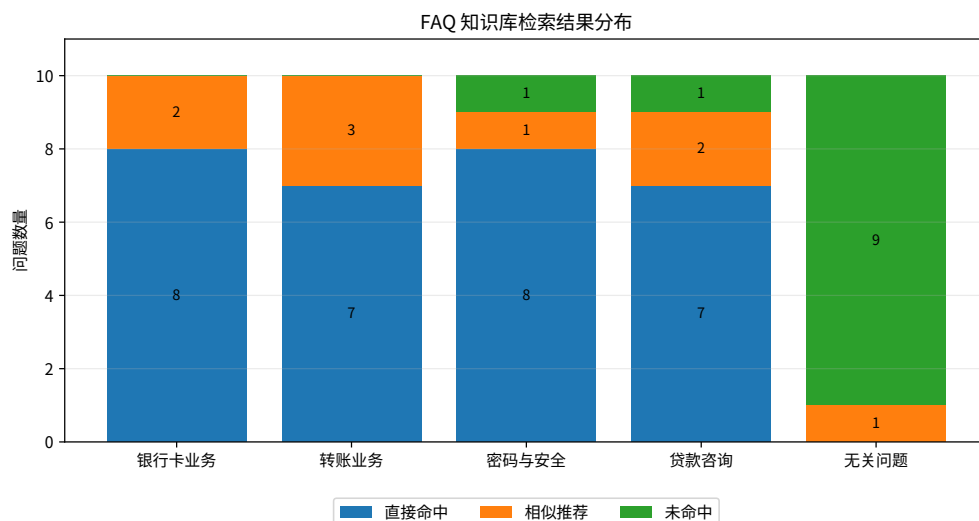


图 7.3 FAQ 知识库检索结果分布

从图 7.3 可以看出，银行卡业务、转账业务、密码与安全、贷款咨询等银行相关问题主要进入直接命中或相似推荐分支，无关问题大多进入未命中分支。这说明 FAQ 知识库既能补充常见咨询问答，也能在一定程度上避免系统对非银行问题直接编造答案。结合表 7.7，40 条银行业务相关 FAQ 样本中，系统直接命中 30 条、相似推荐 8 条、未命中 2 条；10 条无关问题中，9 条进入 no_answer。

余额查询测试主要验证账户数据读取链路。用户输入余额查询请求后，后端将其解析为 balance 意图，并调用账户服务读取当前用户账户信息。测试关注两点：一是回复内容不能脱离数据库余额自行生成；二是账户不存在或数据异常时，系统应返回明确提示，而不是给出含糊的正常结果。该测试说明余额查询已具备智能客服基础能力。

流水查询测试主要验证自然语言条件到查询参数的映射。系统需要从“最近 5 条支出记录”“17 号收入记录”等表达中识别交易方向、数量限制和日期信息。对于日期不完整的表达，系统不会直接猜测月份，而是进入多轮补全并要求用户进一步说明。相比直接推断，这种处理更稳妥，能降低查询范围错误带来的误解。从样例运行情况看，常见流水查询表达能够被转换为可执行条件，返回的流水列表也与后端记录保持一致。

账单统计与分析测试主要验证数据库聚合和图表数据生成。系统接收到“统计这个月总支出”“最近 7 天支出趋势”等请求后，会构造 StatisticsQueryDTO，并由后端统计服务完成聚合计算。大模型不直接计算金额，而是在后端已形成事实数据后生成说明性文本。测试中，若统计结果为空，系统也需要说明当前范围内没有相关记录。该设计符合银行业务对数据真实可靠的要求。

周报/月报测试主要验证系统主动概览能力。用户打开 AI 面板后，前端请求简报数据，并把收入、支出、净流入、趋势和快捷追问组织成卡片。该能力的价值在于用户不必主动输入复杂问题，也能快速了解近期账户变化。测试时重点检查统计周期是否正确、卡片数据是否完整，以及快捷追问能否继续进入 AI 聊天流程。

智能转账测试是安全性要求最高的部分。系统需要识别收款人、金额和备注，并依据常用收款人信息匹配真实收款账号。对于 T7 这类已给出明确金额的请求，系统以用户显式输入金额为准，完成收款人匹配、金额解析、备注解析和转账前校验后，只生成待确认信息，不直接扣款。对于 T8 场景，系统能识别取消意图，清除 pendingMap 中的待确认转账状态，保证用户取消后不再保留旧资金操作请求。对于 T9 这类未提供明确金额但带有时间参考的请求，系统才进入历史金额复用逻辑：先根据收款人昵称确定收款账号，再结合备注关键词和参考时间范围查询历史支出流水；若查到匹配记录，则复用历史金额生成确认信息，否则要求用户补充金额。测试结果表明，智能转账在自然语言解析、常用收款人匹配、历史金额复用、确认执行和取消回退等分支上都能保持“确认前不执行”的原则。

7.5 用户体验测试与反馈分析

除功能测试外，本文还通过系统中的匿名反馈测试模块进行了小规模用户体验测试，以回应任务书中“进行用户体验测试，收集用户反馈，并根据反馈优化机器人的交互设计，以提高用户满意度”的要求。测试者在本地运行环境进入 AI 面板的匿名反馈入口，选择匿名角色后按任务卡片完成 U1-U8 八类任务，并在每个任务后填写理解度、满意度、安全感、语言表达四项整数评分和文字反馈。需要说明的是，本节数据来自测试者实际完成本地原型测试后的记录，但系统使用虚拟账户、虚拟流水和演示收款人，不接入真实银行账户和真实客户生产数据。

测试对象按使用背景分为四类，既包括熟悉计算机系统的用户，也包括普通手机银行用户和具有行业经验的指导人员。分组情况如表 7.8 所示。

表 7.8 用户体验测试对象分组

用户群体	人数	测试目的
计算机专业学生	3	验证系统功能逻辑、页面反馈和 AI 处理过程是否容易理解。
普通手机银行用户	3	模拟普通用户使用手机银行客服的场景，观察口语化提问、语音输入和图表理解是否顺畅。
计算机行业开发者	2	从开发者视角检查接口状态、日志记录、转账确认和异常提示是否合理。
银行业务/企业导师	1	从银行业务边界、数据安全说明和论文实验设计角度评估系统完整性。

测试任务围绕系统已经实现的核心功能设计，覆盖查询、统计、FAQ 问答、语音输入、转账确认取消和异常情绪安抚等典型场景。具体任务见表 7.9。

表 7.9 用户体验测试任务

编号	测试任务	观察重点
U1	查询账户余额	用户能否用自然语言得到当前虚拟账户余额，回答是否清楚。
U2	查询最近 5 条支出流水	用户能否理解支出流水列表，系统返回记录条数是否符合预期。
U3	咨询银行卡挂失、冻结等安全问题	FAQ 知识库是否返回银行卡安全处理步骤，用户是否理解处理建议。
U4	咨询转账到账时间	FAQ 是否能回答到账时间，并避免给出绝对化承诺。
U5	使用语音输入查询余额	浏览器语音识别是否能回填输入框，用户是否需要二次修改文本。
U6	查询本月总支出并查看图表	用户是否能同时理解文字统计和 ECharts 图表展示。
U7	发起转账后点击取消	系统是否先展示确认信息，用户是否能明确取消资金操作。
U8	投诉或强烈不满表达	系统是否触发情绪安抚，并引导用户联系人工客服。

匿名反馈测试模块会在 assistant_test_log 表中写入三类记录：feedback_session_start 表示测试开始，assistant_chat_test 表示测试者完成某一任务时的聊天日志，feedback_session_end 表示测试结束和总体反馈。前端提交 test_session_id、participant_code、participant_group、task_code、task_name、operation_steps、四类评分和 user_feedback；后端依据实际处理结果写入 intent、trace_tool、trace_status、success、completed、duration_ms 等字段。日志字段与原始输入节选

见附录 A.5。

本次统计使用 assistant_test_log 表中 source=usability_live 且 participant_code 为 P1–P9 的记录。记录时间范围为 2026 年 4 月 21 日 12:40:00 至 2026 年 4 月 29 日 10:13:30，共 99 条日志，其中开始记录 9 条、测试聊天记录 81 条、结束反馈记录 9 条。U7 “转账后取消” 由 “发起转账” 和 “取消转账” 两条聊天日志组成，因此 8 个任务对应 9 条任务交互日志。统计口径为：完成率等于 completed 字段求和除以记录数，平均响应耗时等于 duration_ms 字段均值除以 1000，平均操作步数取 operation_steps 字段均值，理解度、满意度、安全感和语言表达分别取对应评分字段均值。评分采用 1–5 分，分值越高表示体验越好。

用户体验测试结果按任务完成率、平均响应耗时、平均操作步数和平均评分统计。其中，完成率由 completed 字段计算，响应耗时由 duration_ms 字段换算得到，操作步数来自 operation_steps 字段，理解度、满意度、安全感和语言表达由测试者 1–5 分评分求均值得到。

基于上述记录得到的用户体验测试结果见表 7.10。

表 7.10 用户体验测试结果汇总表

编号	任务名称	记录数	完成率	响 应 耗 时/s	步数	理解度	满意度	安全感	语 言 表 达
U1	查询账户余额	9	100.0%	2.72	2.3	4.7	4.3	4.7	4.1
U2	查询支出流水	9	100.0%	2.66	3.3	4.0	3.7	4.0	3.7
U3	银行卡安全 FAQ	9	100.0%	2.56	3.0	4.2	4.0	5.0	4.0
U4	转账到账 FAQ	9	100.0%	2.61	3.0	4.0	4.0	4.0	4.0
U5	语音查询余额	9	100.0%	2.74	3.9	3.7	3.7	3.7	3.7
U6	支出统计图表	9	100.0%	6.90	3.3	3.8	3.7	4.0	3.7
U7	转账后取消	18	100.0%	1.48	2.2	4.6	4.2	5.0	4.1
U8	投诉情绪安抚	9	100.0%	0.18	3.0	4.1	4.0	5.0	4.0

以 U6 “支出统计图表” 任务为例，该任务共有 9 条测试记录且全部完成，因此完成率为

$$C_{U6} = \frac{9}{9} \times 100\% = 100.0\%.$$

该任务平均响应耗时为 6.90s，明显高于其他任务，说明统计聚合、图表数据组织和说明文本生成会增加处理时间。再以 U7 “转账后取消” 为例，该任务由发起转账和取消转账两条交互组成，共 18 条记录且全部完成，因此完成率为

$$C_{U7} = \frac{18}{18} \times 100\% = 100.0\%.$$

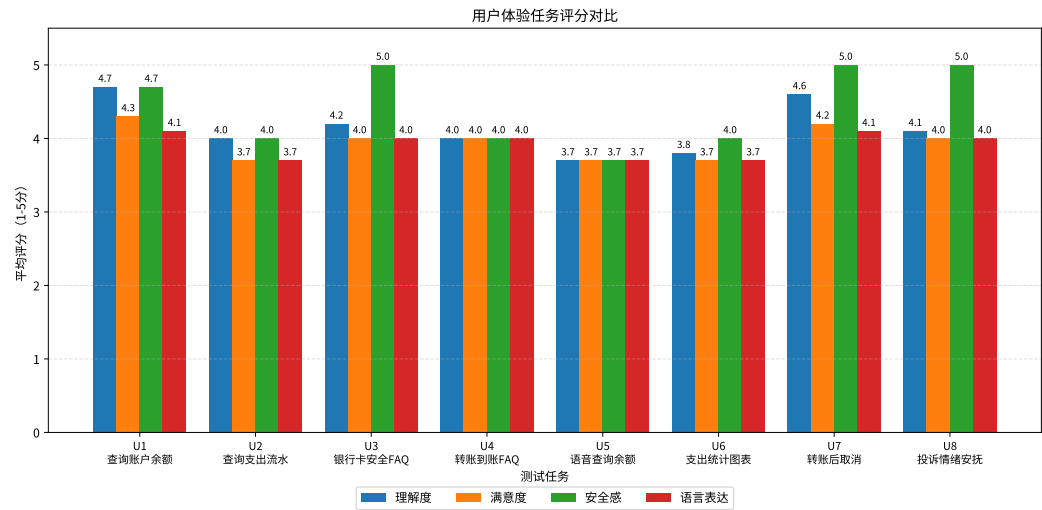


图 7.4 用户体验任务评分对比

图 7.4 对比了不同用户体验任务的理解度、满意度、安全感和语言表达评分。余额查询、转账取消和投诉情绪安抚评分较高，说明用户对查询类和安全确认类交互更容易理解。语音查询和支出统计图表评分相对较低，反映出语音环境、权限提示、统计口径说明、图表解释和回复表达仍有优化空间。

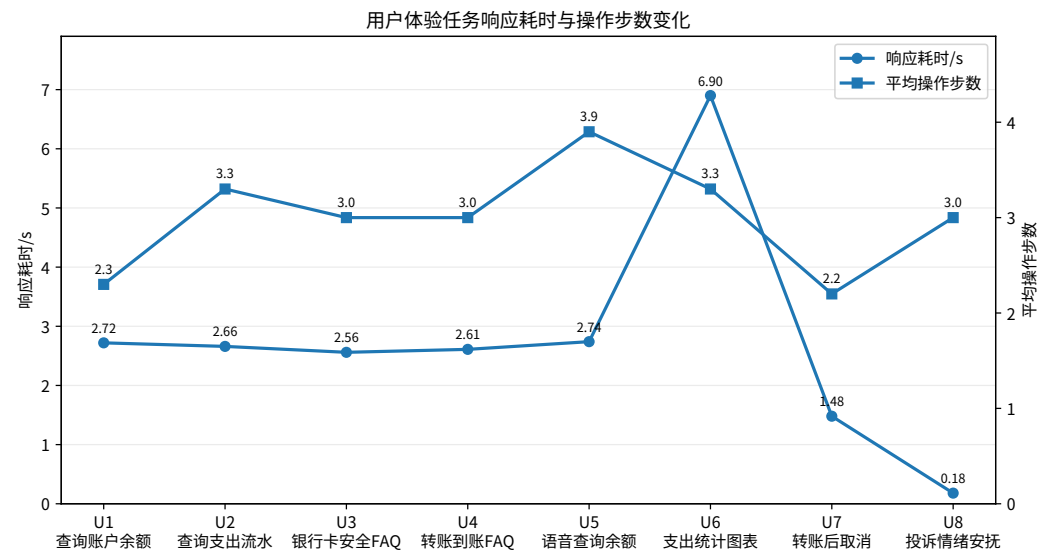


图 7.5 用户体验任务响应耗时与操作步数变化

图 7.5 展示不同体验任务的平均响应耗时和平均操作步数。U6 支出统计图表响应耗时明显高于其他任务，主要原因是该任务需要完成统计聚合、图表数据组织和说明文本生成。U7 转账取消和 U8 投诉情绪安抚耗时较低，说明规则分支和待确认状态处理响应较快。结合表 7.10，U2 支出流水查询虽能完成，但普通用户反馈中出现“筛选词不太会说”“希望默认突出支出”等意见，说明自然语言提示和结果展示还可继续简化。

按测试对象类型汇总后的结果见表 7.11。不同群体的评分差异与测试观察基本一致：计算机专业学生和开发者更关注状态流转与日志可追溯性；普通手机银行用户更关注需求是否容易说清、图表是否能看懂以及语音输入是否稳定；行业导师更关注业务边界和安全说明。

表 7.11 不同测试对象的反馈汇总

用户群体	人数	聊天记录	满意度	安全感	语言	主要反馈
计算机专业学生	3	27	4.1	4.6	4.1	流程顺畅，口语输入基本能命中，统计解释可再简化。
普通手机银行用户	3	27	3.7	4.3	3.6	能完成主要操作，但语音权限、噪声和图表理解存在门槛。
计算机行业开发者	2	18	4.0	4.6	4.0	状态流转、日志字段、FAQ 边界和取消链路较清楚。
银行业务/企业导师	1	9	4.2	4.6	4.0	业务边界、风险提示和实验口径基本一致，建议强化口径说明。

为了回应“提高客户服务效率，减少人工客服负担”的目标，本文根据实际用户体验反馈，将传统页面操作路径与 AI 助手操作路径进行对照。该对照不用于夸大系统性能，而是从页面跳转、筛选设置、人工咨询和确认步骤角度，说明智能客服入口对高频业务的路径压缩作用，结果见表 7.12。

表 7.12 传统页面操作与 AI 助手操作路径对照

任务	传统页面操作	AI 助手操作	优化效果
查询最近 5 条支出流水	进入流水页，选择支出类型，设置时间或条数条件后查询。	输入“查最近 5 条支出记录”或相近口语表达。	操作步数减少。
查询本月总支出	打开流水页，筛选本月支出记录，再查看图表或人工汇总。	输入“统计这个月总支出”，由后端生成统计和图表。	查询路径缩短。
银行卡安全咨询	查找菜单、帮助文档或转人工客服咨询挂失、冻结等处理流程。	输入“卡丢了怎么办”等问题，由 FAQ 知识库返回标准答案或推荐。	常见问题直接回答。
转账取消	进入转账页，填写收款人与金额信息后返回或取消。	AI 生成确认信息后点击取消按钮。	降低误操作风险。

从表 7.12 可以看出，AI 助手不是替代全部传统页面，而是在用户目标明确、操作路径较长或问题高频重复的场景中提供更短入口。对于流水查询和账单统计，系统把页面筛选条件转换为自然语言意图和后端查询参数；对于银行卡挂失等 FAQ 咨询，系统优先通过知识库回答，减少用户查找菜单或等待人工客服的可能；对于转账取消，系统把高风险操作停留在确认阶段，并提供取消按钮，使用户能在执行前明确终止操作。因此，本文的效率提升主要体现在查询路径缩短、重复咨询分流和风险操作确认三个方面。

从反馈分析角度看，用户体验测试的价值不只是得到评分，还在于把用户反馈转化为具体优化措施。结合 assistant_test_log 中的任务反馈和结束反馈，本文将

主要反馈问题与对应优化措施整理为表 7.13。

表 7.13 用户反馈问题与对应优化措施

用户反馈问题	对应优化措施
普通用户不确定该如何描述支出筛选条件	在任务卡和快捷追问中增加“查最近 5 条支出”“本月支出汇总”等示例，引导用户使用更明确的表达。
语音输入受浏览器权限、环境噪声和转写误差影响	保留语音转写后的文本确认步骤，并在语音失败时允许用户直接改为文字输入。
统计图表需要配合文字解释才能理解	在图表上方增加统计口径、时间范围和总额说明，减少用户单独理解图表的成本。
转账流程担心误操作	增加确认/取消按钮和账号脱敏展示，在用户明确确认前不执行扣款。
常见业务问题不想查菜单	增加 FAQ 知识库问答，并通过 <code>faq_query_log</code> 记录未命中和推荐问题，便于后续补充知识库。
投诉或焦虑问题不适合普通回答	增加情绪安抚和人工客服引导，避免系统继续输出普通闲聊式回答。

匿名用户体验测试进一步观察了用户是否能顺利理解和使用这些功能，使实验部分能更完整地支撑本文的系统设计目标。

7.6 交互优化设计分析

第一，系统通过流式响应降低等待感。如7.6界面图所示，前端调用 `/assistant/chat/stream` 后，后端先返回 `trace` 事件，提示“请求已接收”和“正在识别业务意图”，最后再返回 `message` 事件。与普通请求一次性返回结果相比，流式处理可以让用户看到系统正在处理请求，减少等待过程中的不确定性。

第二，系统通过展示 AI 处理过程提升可解释性。如7.6界面图所示，`AssistantChatResponse` 中的 `trace` 字段包含 `intent`、`tool`、`status`、`confidence` 和 `steps`。前端把这些信息展示在聊天气泡中，使用户和答辩评阅者能看到系统识别了什么意图、调用了什么能力以及当前处理状态。

第三，系统用图表增强统计结果表达。如7.7界面图所示，趋势类统计不仅返回文字总结，还会显示柱状图或折线图；收支概览则展示收支对比和按对方账号聚合的饼图。图表展示降低了用户理解复杂流水数据的成本。

第四，系统提供语音输入以减少文字输入成本。用户点击语音输入按钮后，可以直接说出查询或转账需求，前端再将语音识别结果填入输入框。该设计在演示场景中较直观，也使语音识别要求落到实际交互功能中。

第五，系统提供情绪安抚与人工客服引导。对于明显包含投诉、焦虑或愤怒

的输入，系统不继续普通闲聊，而是先返回安抚说明，并提示用户拨打虚拟客服电话 010-1234-5678 联系人工客服。该设计使智能客服在业务查询之外，也具备基本客服分流意识。

第六，系统提供 FAQ 知识库问答。银行卡挂失、密码找回、转账到账时间等常见咨询，优先从数据库知识库返回标准答案；未达到直接命中阈值时，系统根据匹配得分给出相似问题并提示用户继续提问，同时记录查询日志，便于后续分析高频问题和未收录问题。

第七，系统通过快捷操作减少输入成本。简报卡片提供快捷追问按钮，用户可以一键继续查询；转账确认场景显示“确认”和“取消”按钮，减少用户手动输入。

第八，系统对不支持能力进行边界控制。当前项目没有实现消费品类、商户类型和场景标签分析，因此后端在意图解析后会拦截此类需求，避免大模型给出超出系统能力的结论。该设计有助于提高银行场景下的可信度。



图 7.6 AI 处理过程展示界面图

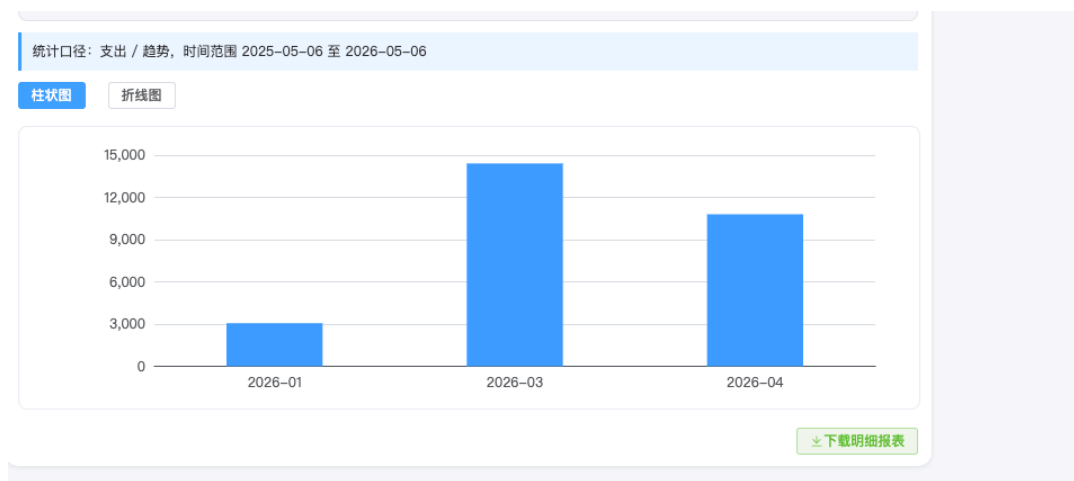


图 7.7 AI 统计图表展示界面图

7.7 测试结论

综合功能测试和交互分析，BankCopilot 能够完成基于文本输入和语音转文字输入的银行业务交互。查询类任务中，系统可以正确调用账户和流水服务；统计类任务中，系统可以基于数据库聚合结果生成图表和分析；FAQ 咨询中，系统可以从知识库返回标准答案或相似问题推荐；转账类任务中，系统通过二次确认保证资金操作安全。虽然当前核心功能数量有限，但链路完整、场景典型，整体上体现了智能客服机器人在银行场景中的可行性和实用价值。

从测试结论看，本文系统已经形成从自然语言输入、业务服务调用、知识库检索到结构化结果展示的闭环。优势主要有三点：一是系统没有把银行业务结果和常见问题答案完全交给大模型生成，而是以数据库、知识库表和后端服务为依据；二是系统对查询、统计、FAQ 和转账采用不同交互策略，能根据业务风险调整流程复杂度；三是展示层保留 AI 处理过程和统计口径说明，便于用户理解系统为何给出某一结果。

同时，受项目周期、原型系统定位和企业业务数据保密要求限制，本文测试未接入真实银行生产数据，也未覆盖真实银行系统中的复杂账户状态、异常交易、并发转账和风控规则。当前测试数据规模较小，FAQ 知识库条目数量有限，自动化测试主要覆盖服务层核心分支。若将系统扩展到更真实的业务环境，还需要补充接口集成测试、压力测试、安全测试和异常恢复测试。本文在结论中将这些问题作为不足说明，以避免夸大系统当前能力。

第八章 结论与展望

8.1 主要结论

本文围绕“智能客服机器人的设计与交互优化”完成了 BankCopilot 银行智能客服机器人原型的设计与实现。作为本科计算机专业毕业设计，本系统并不定位为完整商业银行系统，而是在规模可控的银行管理系统中，验证 AI 能力如何接入传统业务管理流程。系统功能不追求大而全，而是围绕语音输入、FAQ 知识库问答、异常情绪安抚、余额查询、流水查询、账单统计、周报/月报和智能转账等典型场景，形成一个小而完整、边界清晰、可以演示的 AI 业务闭环。

从功能设计看，各项 AI 能力承担的业务角色并不重复。语音输入扩展了自然语言入口，用户可以直接说出需求，再由前端转写为文本进入同一聊天链路。FAQ 知识库问答处理银行卡挂失、转账到账时间、密码找回等常见咨询，答案来自数据库知识库表。情绪安抚用于识别客服场景中的明显负面情绪，并返回安抚话术和虚拟客服电话。AI 查余额体现自然语言到数据库单对象查询的转换，用户用口语化方式提出问题，后端仍从账户表读取真实余额和账户状态。AI 查流水体现自然语言条件到列表查询条件的转换，需要识别交易方向、条数限制和时间范围。AI 账单统计与分析体现数据库聚合能力，把流水数据转换为总额、笔数、趋势和对比结果。AI 周报/月报是在统计结果基础上的概览展示，强调主动汇总和快捷追问。AI 智能转账则对应流程型操作，需要按收款人匹配、金额解析、业务校验、待确认、确认执行或取消的步骤完成。

从系统实现看，AI 与传统管理系统结合的关键，不是让模型直接完成所有业务，而是明确模型和后端系统的职责边界。大语言模型适合自然语言理解、意图识别和结果表达，但账户数据、流水数据、统计结果和转账执行必须由 Spring Boot 后端和 MySQL 数据库提供确定性支持。对于银行业务这类需要准确性和可追溯性的场景，时间范围、统计指标、交易方向、金额、收款人和确认状态等条件都必须被明确约束，不能交给模型自由推断。

从论文创新性看，BankCopilot 的创新不在于提出新的大模型算法，而在于围绕银行业务原型完成系统设计和工程集成。第一，系统面向银行高频业务构建自然语言辅助操作流程，把余额查询、流水查询、账单统计、FAQ 咨询和转账确认统一接入 AI 助手，使传统页面操作扩展为文本和语音转文字驱动的智能交互。第二，系统设计了大语言模型与确定性业务服务分离的集成方式，大模型只负责意图解析和分析性表达，账户、流水、统计和转账结果均由后端服务和数据库生成，

从而降低模型幻觉对业务事实的影响。第三，系统围绕银行业务安全和用户理解成本，引入处理过程展示、统计口径说明、FAQ 日志、异常情绪分流、转账二次确认、取消回退和历史金额复用等机制，使智能客服具备可解释、可控制和可验证的业务辅助能力。

8.2 实践价值

BankCopilot 的实践价值集中体现在智能客服能力与银行业务流程的结合。传统页面适合结构化操作，但当用户目标明确且路径较复杂时，自然语言入口可以减少页面查找和条件设置成本。例如，用户输入“查最近 5 条支出记录”后，系统即可完成意图识别、条件转换、流水查询和结果展示。对于账单统计，系统可以把分散流水转化为总额、趋势和概览，帮助用户更快理解账户变化。

安全性方面，本文系统没有因为引入 AI 而弱化业务控制。尤其在转账功能中，系统把自然语言解析结果转为待确认信息，并在确认前保持状态而不执行资金变动。这样既保留智能交互的便利性，又符合资金类业务需要用户明确授权的原则。对于毕业设计展示，该功能也能较直观地体现：智能并不等于自动越权执行，而是要在用户授权、业务规则和交互体验之间保持平衡。

可解释性方面，系统通过 trace 信息展示意图图、工具、状态、置信度和处理步骤，使 AI 处理过程从黑盒回复变成可观察链路。用户可以理解系统处理逻辑，教师评阅时也能确认系统不是简单静态问答。对于账单分析和统计图表，系统同时给出统计口径说明，避免用户混淆不同时间范围、交易方向或聚合方式下的结果。

8.3 不足与展望

受本人开发经验、项目周期和测试条件限制，当前系统仍有不足。首先，多轮对话状态主要保存在后端内存中，服务重启后会丢失，这种方式适合毕业设计原型展示，但距离长期稳定运行还有差距。其次，系统依赖外部大模型服务，当网络异常或模型接口不可用时，意图解析和分析总结能力会受到影响，虽然已有部分规则化后备处理，但仍不够完善。再次，当前交易流水字段较简单，系统可以完成余额查询、流水查询、统计分析和转账确认，但还不能深入分析消费品类、商户类型和具体消费场景。

后续若继续完善该系统，可以从几个较实际的方向入手。第一，将待确认转账、待补全查询等会话状态保存到 Redis 或数据库中，提高服务重启后的恢复能力。第二，继续补充异常处理和测试用例，例如模型返回格式异常、网络请求失

败、余额不足、收款人不存在等场景，使系统在演示之外更加稳定。第三，在交易流水中增加更丰富的字段，并在此基础上扩展消费分类、预算提醒和更细粒度的账单分析功能。经过这些改进，BankCopilot 可以在现有毕业设计原型基础上进一步完善；就本次毕业设计而言，系统已经完成 AI 与传统银行管理系统结合的主要设计目标。

第九章 结束语

本文围绕个人银行业务中的智能客服交互问题，完成了从需求分析、总体设计、核心功能实现到测试分析的论述。论文写作始终以本人独立完成的项目源码为依据，只说明系统中已经实现并可以演示的功能，没有把未完成的能力写成系统成果。系统重点不是构建泛化聊天机器人，而是在一个具体银行管理系统中，验证大语言模型与确定性业务服务结合的实现方式。

从实现结果看，BankCopilot 已能处理语音输入、FAQ 知识库问答、异常情绪安抚、余额查询、流水查询、账单统计、周报/月报和智能转账等典型场景。系统通过语音转文字和自然语言意图识别降低用户操作门槛，通过数据库知识库提供常见问题标准答案，通过规则化情绪检测补充客服安抚与人工引导，通过后端业务服务保证数据来源和资金操作确定性，并通过处理过程展示、统计口径说明和转账二次确认增强交互可解释性与可控性。虽然系统仍存在测试数据规模有限、风控规则较简单、会话状态未持久化等不足，但作为本科毕业设计原型，已经较完整地展示 AI 能力嵌入传统管理系统后的基本流程和实现效果。

通过完成本课题，本人对前后端分离开发、数据库建模、接口设计、自然语言意图识别、流式交互和 LaTeX 论文写作有了更系统的认识。开发和论文整理过程也让我认识到，AI 功能落地到具体业务系统时，不能只关注模型回复是否自然，更要关注数据来源、业务约束、异常处理和用户确认等工程细节。本次毕业设计仍有不足，但基本达到了任务书对系统设计、功能实现和论文撰写的要求。

致谢

本论文和系统原型的完成，离不开指导教师和企业专家导师在选题方向、系统设计和论文写作方面给予的指导。项目实施过程中，相关课程学习和开源技术文档也为系统架构设计、前后端开发和数据库实现提供了帮助。同时，感谢毕业设计过程中给予建议和支持的指导教师、企业专家导师、辅导员和同学。由于本人能力和时间有限，论文和系统仍存在不足，恳请各位批阅老师批评指正。

参考文献

- [1] ZHAO W X, ZHOU K, LI J, et al. A survey of large language models[A/OL]. arXiv (2023). <https://arxiv.org/abs/2303.18223>.
- [2] ALCAZAR J, BAIRD S, CRONENWETH E, et al. Core banking systems and options for modernization[R]. Kansas City: Federal Reserve Bank of Kansas City, 2024.
- [3] CRISANTO J C, LEUTERIO C B, PRENIO J, et al. Regulating AI in the financial sector: recent developments and main challenges: FSI Insights No. 63[R]. Basel: Bank for International Settlements, 2024.
- [4] Bank of America. A decade of AI innovation: BofA's virtual assistant erica surpasses 3 billion client interactions[M/OL]. Charlotte: Bank of America Newsroom, 2025. <https://newsroom.bankofamerica.com/content/newsroom/press-releases/2025/08/a-decade-of-ai-innovation--bofa-s-virtual-assistant-erica-surpas.html>.
- [5] JPMorgan Chase & Co. 2016 annual report[M/OL]. New York: JPMorgan Chase & Co., 2017. <https://www.jpmorganchase.com/content/dam/jpmc/jpmorgan-chase-and-co/investor-relations/documents/2016-annualreport.pdf>.
- [6] China Merchants Bank. 2024 annual report[M/OL]. Shenzhen: China Merchants Bank, 2025. <https://www.cmbchina.com/cmbir/intro.aspx?type=report&version=B>.
- [7] National Institute of Standards and Technology. Artificial intelligence risk management framework: AI RMF 1.0: NIST AI 100-1[R]. Gaithersburg: National Institute of Standards and Technology, 2023.
- [8] International Committee on Credit Reporting. Credit scoring approaches guidelines[R]. Washington, DC: World Bank Group, 2020.
- [9] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[C]//Advances in Neural Information Processing Systems: Vol. 30. 2017.
- [10] BROWN T B, MANN B, RYDER N, et al. Language models are few-shot learners[C]//Advances in Neural Information Processing Systems: Vol. 33. 2020: 1877-1901.
- [11] Alibaba Cloud. DashScope model service documentation[M/OL]. Hangzhou: Alibaba Cloud, 2025. <https://help.aliyun.com/zh/model-studio/>.
- [12] Spring Team. Spring boot reference documentation[M/OL]. Palo Alto: VMware, 2025. <https://docs.spring.io/spring-boot/index.html>.
- [13] Vue.js Team. Vue.js guide[M/OL]. Online: Vuejs.org, 2025. <https://vuejs.org/guide/>.
- [14] Apache ECharts Team. Apache ECharts documentation[M/OL]. Wilmington: Apache Software Foundation, 2025. <https://echarts.apache.org/>.
- [15] MyBatis Team. MyBatis 3 user guide[M/OL]. Online: MyBatis.org, 2025. <https://mybatis.org/mybatis-3/>.

附录 A 主要接口与数据结构

A.1 AI 聊天请求结构

AI 聊天请求由 AssistantChatRequest 承载，核心字段为 message。用户的自然语言输入会以 JSON 请求体形式发送至 /assistant/chat 或 /assistant/chat/stream 接口。

表 A.1 AssistantChatRequest 请求字段

字段	类型	说明
message	String	用户自然语言输入，例如“统计这个月总支出”。

A.2 AI 聊天响应结构

表 A.2 AssistantChatResponse 响应字段

字段	类型	说明
reply	String	AI 回复文本。
chartType	String	图表类型，如 number、bar、line、overview。
chartData	Object	图表数据或数字统计结果。
exportUrl	String	导出交易流水的链接。
trace	TraceInfo	AI 处理过程信息。
explain	Map	查询或统计口径说明。

A.3 统计查询结构

表 A.3 StatisticsQueryDTO 字段说明

字段	说明
accountId	账户 ID。
txnType	收入或支出。
metric	统计指标，支持 sum、count、max、trend。
groupBy	分组方式，支持 none、day、month。
startTime	统计开始时间。
endTime	统计结束时间。
rangeValue	最近 N 天或 N 年的数值。
comparePrevious	是否对比上一周期。

A.4 测试样本说明

为避免把少量功能用例误写成开放域测试，本文在附录中补充意图识别和 FAQ 知识库测试样本的构成说明。意图与分流测试样本围绕系统已实现的基础意图类型和服务编排分支设计，每类 10 条，共 80 条，样本构成见表 A.4。

表 A.4 意图与分流测试样本构成

测试类型	样本数量	典型样例
balance 基础意图	10	查一下我的余额；我现在账户里还有多少钱。
transactions 基础意图	10	查最近 5 条支出记录；17 号收入记录给我看看。
statistics 基础意图	10	统计这个月总支出；最近 7 天支出趋势。
transfer 基础意图	10	给弟弟转 300 元午餐费；给弟弟转晚餐费，和昨天一样。
FAQ 咨询分流	10	卡丢了怎么挂失；跨行转账要多久；验证码收不到怎么办。
情绪关键词后备处理	10	我很生气，我要投诉；我有点担心这笔钱；服务太差了。
不支持能力拦截	10	按消费场景标签统计一下；按商户类型分析消费；给我推荐理财产品。
多轮与确认状态	10	1 月；确认；取消；不转了；重新查今年 1 月收入。

FAQ 知识库测试样本来自项目中的 sql/faq_test_dataset.csv，覆盖标准 FAQ 相似问法、模糊问法和无关问题三类，共 60 条，样本构成见表 A.5。第七章统计表从该数据集中选取 50 条，并按业务类别重新分组统计。

表 A.5 FAQ 知识库测试样本构成

样本类型	数量	期望结果	典型样例
标准 FAQ 相似问法	40	hit	卡丢了怎么挂失；跨行转账要多久；短信验证码不来。
模糊问法	10	recommend	转账有问题；密码怎么办；贷款怎么办。
无关或未收录问题	10	no_answer	今天上海天气怎么样；帮我订一张机票；写一首诗。

A.5 匿名反馈测试日志明细

本附录补充第七章用户体验测试的数据来源说明。匿名反馈测试记录保存在 assistant_test_log 表中，source 字段取值为 usability_live。测试时间范围为 2026 年

4 月 21 日 12:40:00 至 2026 年 4 月 29 日 10:13:30。相关数据由测试者在本地原型环境完成匿名反馈测试后形成，账户、流水和转账对象均为系统演示数据，不包含真实银行客户生产数据。

表 A.6 匿名反馈测试日志记录类型

场景类型	记录数	说明
feedback_session_start	9	记录测试开始时间、匿名角色、participant_code 和 test_session_id。
assistant_chat_test	81	记录 U1-U8 任务中的用户原始输入、系统识别意图、调用工具、响应耗时、操作步数、四类评分和任务反馈。
feedback_session_end	9	记录测试结束时间和测试者对整个匿名反馈测试过程的总结性反馈。

表 A.7 匿名反馈测试对象明细汇总

编号	分组	聊天记录	任务数	响应/s	满意度	安全感	结束反馈摘要
P1	计算机专业学生	9	8	2.34	4.2	4.6	流程顺，转账前确认和取消让用户有底。
P2	计算机专业学生	9	8	2.40	4.0	4.6	口语输入基本能命中，统计解释还能再短一点。
P3	计算机专业学生	9	8	2.53	4.2	4.6	查询与投诉引导在线，语音入口比较好找。
P4	普通手机银行用户	9	8	2.73	3.7	4.3	能完成主要操作，但模糊问法下需要更直白提示。
P5	普通手机银行用户	9	8	2.91	3.7	4.3	语音权限和图表理解有门槛，文字总结更关键。
P6	普通手机银行用户	9	8	3.00	3.7	4.3	噪声环境下语音容易偏，其他流程基本可完成。
P7	计算机行业开发者	9	8	2.37	4.0	4.6	状态流转、日志字段和取消链路清晰，可用于排查。
P8	计算机行业开发者	9	8	2.45	4.0	4.6	查询与转账确认链路稳定，FAQ 边界表达合理。
P9	银行业务/企业导师	9	8	2.61	4.2	4.6	业务边界、风险提示与论文实验口径一致，建议继续强化口径说明。

表 A.8 匿名反馈测试原始输入与反馈节选（3 轮 U1-U8）

编号	任务	原始输入	任务反馈
P1	U1	兄弟，帮我瞅一眼卡里还剩多少钱	余额结果直给，像平时查卡余额一样顺手。
P1	U2	把我最近花出去的 5 笔甩出来	能看懂，但要是默认突出支出会更省脑子。
P1	U3	卡可能丢了，我先干啥，挂失还是冻结	FAQ 能先给步骤，再补风险说明，顺序对。
P1	U4	今晚跨行转账，最晚啥时候到	到账时效讲得比较稳，没有乱承诺。
P1	U5	语音输入：我卡里还剩多少	语音转文字后还能命中余额查询。
P1	U6	本月我花了多少，顺便给个图	总额和图能对应上，解释再多一句更好。
P1	U7	给弟弟转 300，备注午饭	先弹确认再转账，这点很稳。
P1	U7	算了先撤销这笔	取消提示明确，没有真实扣款风险。
P1	U8	这回答让我有点上头，我要投诉	先安抚再引导人工，节奏没问题。
P4	U1	我想看看账户里钱还够不够	能查到余额，但我得看两遍才反应过来。
P4	U2	最近花的钱大概啥情况，给我看下	我说得比较模糊，系统还能给出结果。
P4	U3	银行卡找不到了我该先做哪一步	先挂失再处理的提示让我安心。
P4	U4	跨行转账一般什么时候能到呀	时间说法比较日常，我能理解。
P4	U5	语音输入：我这个账户还有钱吗	语音能用，但有时要重新说一遍。
P4	U6	这个月总共花了多少钱，图也看一下	图表有帮助，不过解释再简单点更好。
P4	U7	给弟弟转三百块午饭钱	先确认再执行，这一步很必要。
P4	U7	先别转了，取消掉	取消后我比较放心。
P4	U8	我有点生气，想找人工客服	情绪场景有安抚，不会继续答非所问。
P7	U1	查询 demo_account 可用余额，返回金额与更新时间	返回字段稳定，余额来源可追溯。
P7	U2	按 direction=OUT 查询最近 5 条交易，按时间倒序	排序和方向过滤符合预期。
P7	U3	银行卡遗失场景，请返回挂失与冻结的处理顺序	知识库命中准确，动作顺序可执行。
P7	U4	说明跨行转账 T+0/T+1 到账条件	FAQ 对边界条件描述清楚。
P7	U5	语音输入：查询当前可用余额	语音只影响输入层，后端链路一致。
P7	U6	统计本月支出并返回图表数据与口径	统计数值与图表一致，可复核。
P7	U7	发起向弟弟转账 300 元，验证待确认状态	transfer_validation 状态流转正确。
P7	U7	取消待确认转账	取消后 pending 状态清理正确。
P7	U8	模拟投诉：回答不满意，请转人工	emotion_support 分流规则可预期。